Apache
Airflow

Training objectives

- Understand the architecture and components of Airflow to orchestrate data workflows.
- Install, configure and start Airflow services and interface.
- Design, implement and manage the execution of DAGs in Airflow.
- Use the Airflow user interface to monitor, manage and execute tasks.
- Schedule and manage task scheduling in Airflow.
- Create workflows with sub-workflows, conditional task management and error handling strategies.
- Build a data processing pipeline using Airflow to extract, clean, transform and load data into a database.
- Implement best practices to optimise the performance and reliability of workflows in Airflow.

Programme

01 **Airflow Fundamentals**

02 **Overview of essential features**

03 **Scheduling and workflows**

04 **Usage and real-world scenarios**

05 **Best practices**

Programme

06 **Integration of external databases**

07 **Scheduling and advanced planning**

08 **Creation and management of complex workflows**

09 **Best practices again**

01

Airflow fundamentals

Introduction

- Apache Airflow: open-source platform enabling the development, scheduling and monitoring of batch-oriented workflows.
- Allows the construction of workflows interconnecting a wide variety of technologies.
- A web interface helps manage the state of workflows.
- Deployable in many ways, from a simple process to a distributed configuration capable of supporting more complex workflows.

Introduction

- In 2014, Airbnb's data team began developing a tool to meet their growing needs for workflow orchestration. They needed a flexible and scalable solution to manage their complex data pipelines, but none of the tools available at the time met their requirements.
- Objective: to create a workflow orchestration platform that is easy to use, flexible, and tailored to the data team's needs. Built as an open-source project from the start, allowing other organisations to benefit from their efforts and contribute to the platform's improvement.
- In 2015, Airflow was made public. Quickly, Airflow attracted the attention of the data community and became a popular tool for workflow orchestration in various companies and organisations.
- In 2016, Airflow was handed over to the Apache Software Foundation: formalising project governance, continued growth as an open-source project. Airflow continued to evolve with an active community of contributors and users (new features, bug fixes, performance improvements…).

Introduction

- Today, Airflow is widely used in many companies for orchestrating data workflows, ETL (Extract-Transform-Load) pipelines…

- Airflow's popularity continues to grow, and it has become an essential part of the data ecosystem for many organisations worldwide.

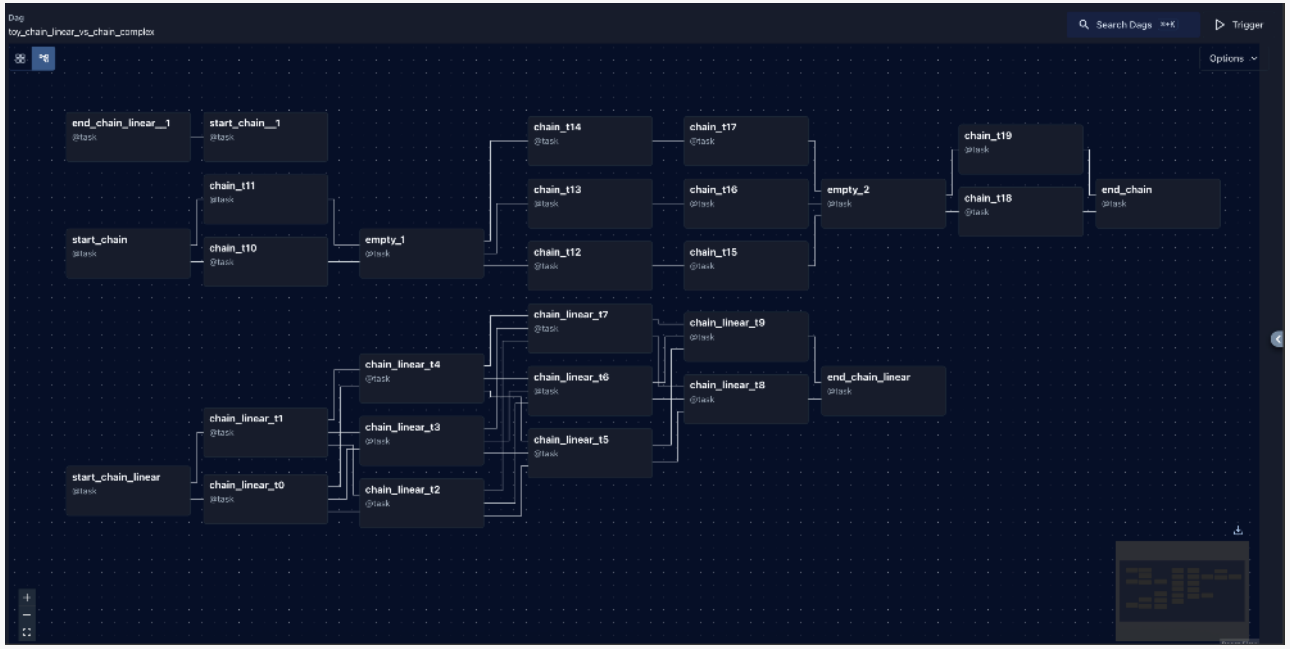
Introduction

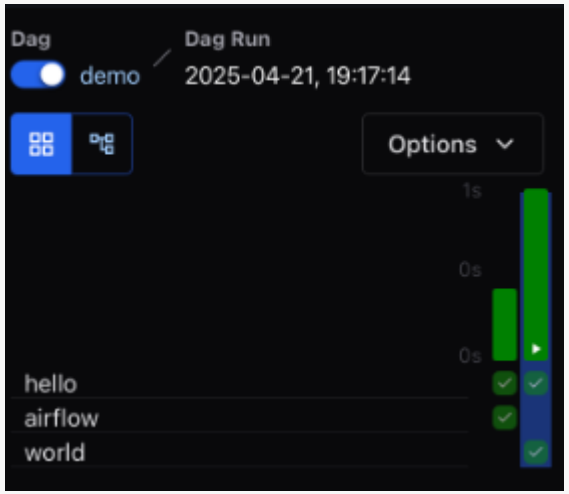
- All workflows are defined in Python code: 'workflow as code'
- Dynamic Airflow pipelines are configured as Python code, allowing for dynamic generation of pipelines.
- Extensible The Airflow framework contains operators to connect to different technologies. Airflow components are extensible to easily adapt to the environment.
- Flexible Workflow parameterisation is integrated using the Jinja templating engine.

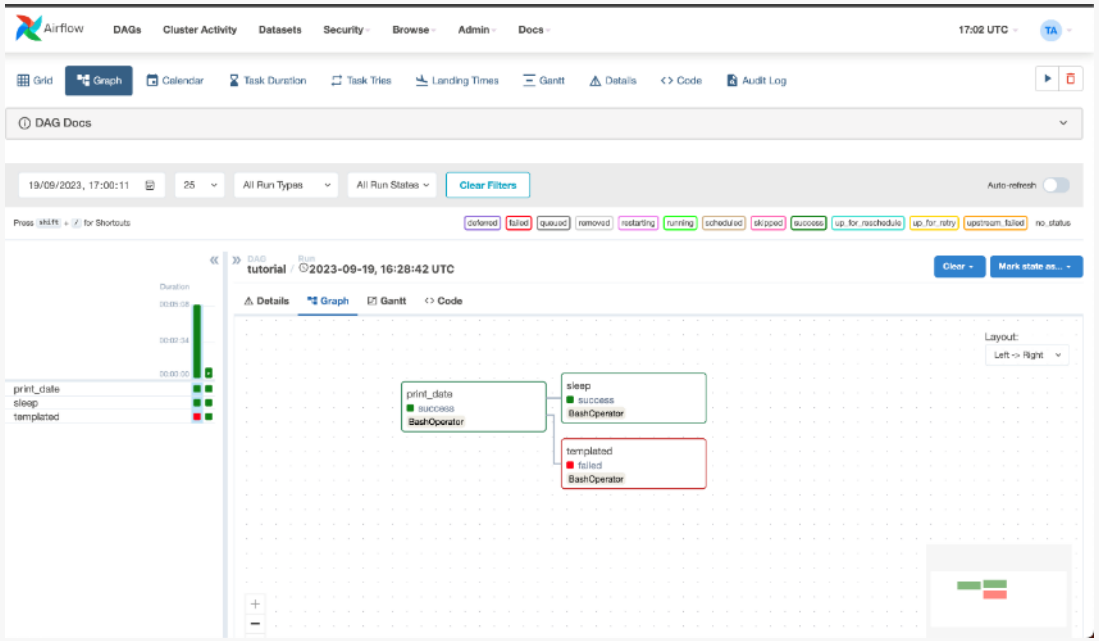
Introduction

```
from datetime import datetime
from airflow import DAG
from airflow.decorators import task
from airflow.providers.standard.operators.bash import BashOperator
# or from airflow.operators.bash import BashOperator

# A DAG represents a workflow, a collection of tasks
with DAG(dag_id="demo", start_date=datetime(2026, 6, 23), schedule="0 0 * * *") as dag:
    # Tasks are represented as operators
    hello = BashOperator(task_id="hello", bash_command="echo hello")

    @task()
    def airflow():
        print("airflow")


    # Set dependencies between tasks
    hello >> airflow()
```

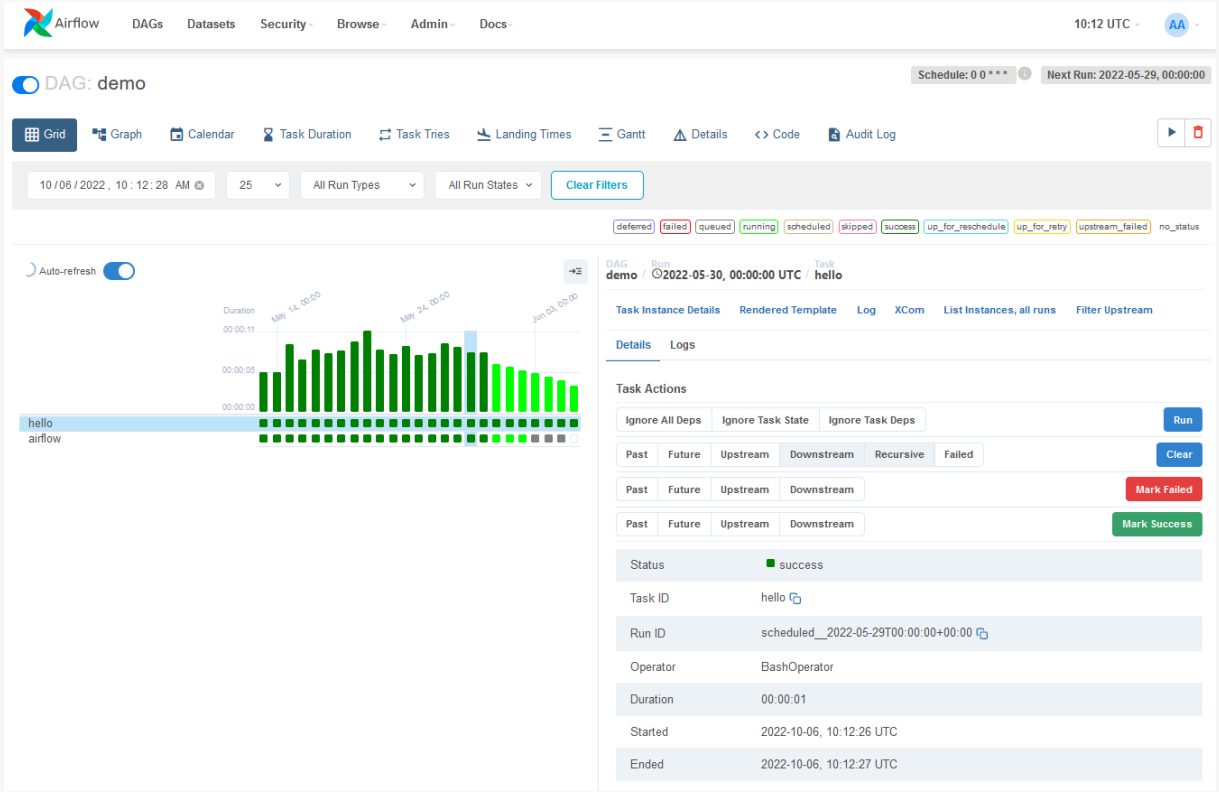
Introduction







Introduction



Introduction

Why use Airflow?

- If a workflow has a clear start and end, and runs at regular intervals, it can be scheduled as an Airflow DAG.
- If you prefer coding rather than clicking.
- Workflows can be stored in version control so you can revert to previous versions. In addition to component extensibility (recall):
  - ➡ Workflows can be developed by multiple people simultaneously.
  - ➡ Tests can be written to validate functionality.
  - ➡ Rich scheduling and execution semantics allow you to easily define complex pipelines running at regular intervals. Backfilling lets you (re)run pipelines on historical data after making changes to your logic. And the ability to rerun partial pipelines after fixing an error helps maximise efficiency.

Introduction

Why use Airflow?

- Airflow User Interface -> detailed views of two elements:
  - Pipelines
  - Tasks

  = an overview of pipelines over time.

  From the interface, you can inspect logs and manage tasks (for example, rerun a task in case of failure).
- **Open-source:** Work on components developed, tested and used by many other companies worldwide.
- **Active community:** Ability to find many useful resources, in the form of blog posts, articles, conferences, books... Channels (Slack), mailing lists...

Airflow as a platform is truly customisable. By using Airflow's public interface, you can extend and customise almost every aspect of Airflow.

Introduction

Why not use Airflow?

- Airflow was designed for finite batch workflows.
- Although the command line interface (CLI) and REST API allow triggering workflows, Airflow was not designed for event-driven workflows running indefinitely. Airflow is not a streaming solution.
- However, a streaming system such as Apache Kafka can be used in collaboration with Apache Airflow. Kafka can be used for real-time ingestion and processing, with event data being written to a storage location (and Airflow periodically starts a workflow to process a batch of data).
- If you prefer clicking rather than coding, Airflow is probably not the right solution. The web interface aims to make workflow management as easy as possible and the Airflow framework is continuously improved to make the developer experience as smooth as possible. But Airflow's philosophy is indeed to define workflows as code, so programming will always be necessary.

Airflow Architecture

Required components

A minimal Airflow installation includes:

- A scheduler

  Manages both the triggering of scheduled workflows and the submission of tasks to the executor to run them. The executor is a configuration property of the scheduler, not a separate component, and runs in the scheduler's process. Several executors are available by default, and it is also possible to write a custom one.

- A web server

  Provides a convenient user interface to inspect, trigger and debug the behaviour of DAGs and tasks.

- A folder

  Containing DAG files, read by the scheduler to determine which tasks to run and when to run them.

- A DB

  A metadata database, used by Airflow components to store the state of workflows and tasks. The configuration of a metadata database is described in the configuration of a Database Backend and is required for Airflow to function properly.

Airflow Architecture

Optional components

Some Airflow components are optional and can allow for better extensibility:

- **Worker**

  Executes the tasks assigned to it by the scheduler. In the basic installation, the worker may be part of the scheduler and not a separate component. It can run as a continuously running process in the CeleryExecutor, or as a POD in the KubernetesExecutor.

- **Triggerer**

  Executes deferred tasks in an asyncio event loop. In the basic installation where deferred tasks are not used, a triggerer is not necessary. More information on task deferral can be found in deferrable operators and triggerers.

Airflow Architecture

Optional components

Some components of Airflow are optional and can allow for better extensibility:

- Dag processor

  Analyses DAG files and serialises them into the metadata database. By default, the DAG processor process is part of the scheduler, but it can run as a separate component for scalability and security reasons. If the DAG processor is present, the scheduler does not need to read DAG files directly.

- Plugins

  Grouped in a folder, plugins are a way to extend Airflow functionality (similar to installed packages). Plugins are read by the scheduler, DAG processor, triggerer, and web server.

Airflow Architecture

Let's emphasise the distinction between worker and executor

- The worker in Airflow is responsible for executing the tasks assigned by the scheduler. It actually runs the task code defined in your DAGs. The worker can be considered as a process or an instance that actively processes tasks. In an Airflow deployment using CeleryExecutor, for example, workers can be Celery processes that execute tasks in parallel.
- The executor, on the other hand, is a configuration in Airflow that determines how tasks are executed. There are several types of executors in Airflow, the most common of which are:

  ➡ SequentialExecutor Executes tasks sequentially in the same process, mainly used for development and debugging.

  ➡ LocalExecutor Executes tasks in parallel in separate processes on the same machine.

  ➡ CeleryExecutor Uses Celery to execute tasks in parallel on distinct workers (Celery processes).

  ➡ KubernetesExecutor Executes tasks in containers on a Kubernetes cluster.

Airflow Architecture

Let’s emphasise the distinction between worker and executor

In summary, a worker is responsible for the actual execution of tasks, whereas the executor determines the mode of task execution in Airflow.

Question 1 - Airflow Components

Which Airflow component stores metadata on the state of workflows and tasks?

- ❑ Web Server
- ❑ Scheduler
- ❑ Worker
- ❑ Database
- ❑ Executor

Question 2 - Airflow Components

Which Airflow component is responsible for the actual execution of tasks?

- ❑ Web Server
- ❑ Scheduler
- ❑ Worker
- ❑ Database
- ❑ Executor

Question 3 - Airflow Components

Which Airflow component is responsible for scheduling and executing tasks according to a defined schedule?

- ❑ Web Server
- ❑ Scheduler
- ❑ Worker
- ❑ Database
- ❑ Executor

Question 4 - Airflow Components

Which Airflow subcomponent determines how tasks are executed?

- ❑ Web Server
- ❑ Scheduler
- ❑ Worker
- ❑ Database
- ❑ Executor

Airflow Fundamentals lab

Installation and configuration of Airflow

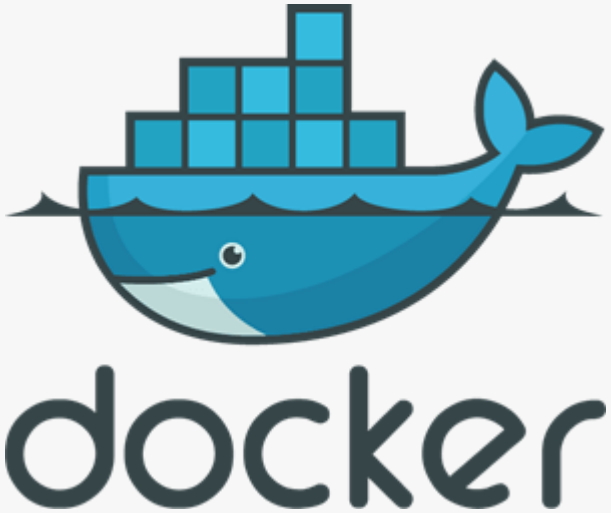
Installation and configuration of Airflow

Docker

Retrieve (manually or using a curl command –LfO) the docker-compose.yaml file at the following address:

`https://airflow.apache.org/docs/apache-airflow/<desired-version>/docker-compose.yaml`

Of course, it is possible to customise this file, but we will use the one from the latest version while explaining the essential components it refers to.



Installation and configuration of Airflow

Overview of environment variables and services

Postgres -

```yaml
services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s
    restart: always
```

PostgreSQL database used as the backend to store Airflow metadata and configuration:

- **Storage of DAG metadata**
- **Tracking execution history**
- **Management of connections and variables**
- **Scheduling and orchestration**
- **Logging and monitoring**

Installation and configuration of Airflow

Overview of environment variables and services

Redis –

```yaml
services:
  redis:
    image: redis:latest
    expose:
      - 6379
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 30s
      retries: 50
      start_period: 30s
    restart: always
```

Message broker for communication between the different Airflow components, such as the scheduler, workers, and flower.

In-memory key-value data storage system, fast and highly scalable, which is used in Airflow for queue management and message transmission between different processes.

Installation and configuration of Airflow

Overview of environment variables and services

The webserver –

```yaml
services:
  airflow-webserver:
    <<: *airflow-common
    command: webserver
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully
```

Airflow web user interface.

Essential component that allows users to view, manage and interact with Airflow workflows and tasks via a user-friendly interface accessible through a web browser.

Installation and configuration of Airflow

Overview of environment variables and services

The scheduler –

```yaml
services:
  airflow-scheduler:
    <<: *airflow-common
    command: scheduler
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8974/health"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully
```

Essential component responsible for scheduling and executing tasks according to a defined timetable.

Installation and configuration of Airflow

Overview of environment variables and services

The Worker –

```yaml
services:
  airflow-worker:
    <<: *airflow-common
    command: celery worker
    healthcheck:
      # yamllint disable rule:line-length
      test:
        - "CMD-SHELL"
        - 'celery --app airflow.providers.celery.executors.celery_executor.app inspect ping -d "celery@$${HOSTNAME}"'
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    environment:
      <<: *airflow-common-env
      # Required to handle warm shutdown of the celery workers properly
      # See https://airflow.apache.org/docs/docker-stack/entrypoint.html#signal-propagation
      DUMB_INIT_SETSID: "0"
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully
```

Optional component responsible for the actual execution of tasks defined in workflows.

Installation and configuration of Airflow

Overview of environment variables and services

The Triggerer –

```yaml
services:
  airflow-triggerer:
    <<: *airflow-common
    command: triggerer
    healthcheck:
      test: ["CMD-SHELL", 'airflow jobs check --job-type TriggererJob --hostname "$${HOSTNAME}"']
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully
```

Responsible for executing deferred tasks in an asyncio event loop.

Used when tasks need to be executed while waiting for an external event.

Installation and configuration of Airflow

Overview of environment variables and services

init –

```
services:
  airflow-init:
    <<: *airflow-common
    entrypoint: /bin/bash
    # yamllint disable rule:line-length
    command:
      - -c
      - |
        if [[ -z "${AIRFLOW_UID}" ]]; then
          echo
          echo -e "\033[1;33mWARNING!!!: AIRFLOW_UID not set!\e[0m"
          echo "If you are on Linux, you SHOULD follow the instructions below to set "
          echo "AIRFLOW_UID environment variable, otherwise files will be owned by root."
          echo "For other operating systems you can get rid of the warning with manually crea
          echo "    See: https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-c
          echo
        fi
        one_meg=1048576
        mem_available=$$(($$(getconf _PHYS_PAGES) * $$(getconf PAGE_SIZE) / one_meg))
        cpus_available=$$(grep -cE 'cpu[0-9]+' /proc/stat)
        disk_available=$$(df / | tail -1 | awk '{print $$4}')
        warning_resources="false"
        if (( mem_available < 4000 )) ; then
          echo
          echo -e "\033[1;33mWARNING!!!: Not enough memory available for Docker.\e[0m"
          echo "At least 4GB of memory required. You have $$(numfmt --to iec $$((mem_availabl
          echo
          warning_resources="true"
```

Responsible for the initialisation of the Airflow database.

When you start or configure Airflow for the first time, you need to run the "init" service to set up and prepare the database required for Airflow to operate.

Installation and configuration of Airflow

Overview of environment variables and services

CLI –

```yaml
services:
  airflow-cli:
    <<: *airflow-common
    profiles:
      - debug
    environment:
      <<: *airflow-common-env
      CONNECTION_CHECK_MAX_COUNT: "0"
    # Workaround for entrypoint issue. See: https://github.com/apache/airflow/issues/16252
    command:
      - bash
      - -c
      - airflow
```

Corresponds to the command line interface provided by Airflow to interact with the Airflow system from the terminal.

Installation and configuration of Airflow

Overview of environment variables and services

Flower –

```yaml
services:
  # You can enable flower by adding "--profile flower" option e.g. docker-compose --profile flower up
  # or by explicitly targeted on the command line e.g. docker-compose up flower.
  # See: https://docs.docker.com/compose/profiles/
  flower:
    <<: *airflow-common
    command: celery flower
    profiles:
      - flower
    ports:
      - "5555:5555"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:5555/"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
      airflow-init:
        condition: service_completed_successfully
```

Monitoring and management tool for Celery tasks, when used as an execution engine in Airflow (which is very often the case).

Installation and configuration of Airflow

Overview of environment variables and services

Celery –

```
AIRFLOW__CORE__EXECUTOR: CeleryExecutor
```

Execution engine (optional) used to run tasks in parallel and in a distributed manner.

Installation and configuration of Airflow

Overview of environment variables and services

Volume postgres-db-volume –

```
volumes:
  postgres-db-volume:
```

Docker volume used to store the data of the PostgreSQL database used by Airflow.

This volume is mounted on the Docker container running the PostgreSQL database service to persistently store database data, such as task metadata, DAGs (Directed Acyclic Graphs), logs, connections…

Using a separate Docker volume for the database allows the database data to be separated from the Docker container image, making it easier to back up, restore, and manage the data independently of the container's lifespan.

Installation and configuration of Airflow

Environment initialization

Before starting Airflow for the first time, you need to prepare your environment, that is, create the necessary files, directories, and initialise the database.

On Linux/WSL, you need to set your host user ID, otherwise, files created in the dags, logs, and plugins folders will be created with the root user. Then, you need to configure them for docker-compose. We will therefore run the following commands:

```
mkdir -p ./dags ./logs ./plugins ./config
echo -e "AIRFLOW_UID=$(id -u)" > .env
```

Installation and configuration of Airflow

Database initialization

You now need to run the database migrations and create the first user account (true for all operating systems). To do this, run:

```
docker compose up airflow-init
```

You should see logs of varying lengths. If everything goes well, they will end with lines similar to these:

```
airflow-init_1 | Admin user airflow created
airflow-init_1 | 2.8.3
start_airflow-init_1 exited with code 0
```

Installation and configuration of Airflow

Created account credentials

The account created in the previous step has the username airflow and the password airflow.

Airflow's default security settings include authentication, which specifically requires users to log in to access the web interface. This is a security measure intended to prevent unauthorised access to Airflow's features and DAGs.

When you access the Airflow web interface for the first time after starting the Airflow server, it will ask you to log in.

Installation and configuration of Airflow

Clean the environment (do not run these commands)

To clean the environment once finished using it or if there is a problem and you want to start from scratch, the simplest way is to follow these two steps:

1. Run the command docker compose down --volumes --remove-orphans in the directory where the docker-compose.yaml file was downloaded.
2. Delete the entire directory in the same folder, using the command rm -rf '<directory>'.

Installation and configuration of Airflow

Start Airflow

It is now possible to start all the services. Run the following command:

```
docker compose up
```

In a second terminal, you can check the status of the containers and ensure that no container is in an abnormal state by running the following command:

```
docker ps
```

```
gitpod /workspace/minimal-airflow-project (main) $ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED         STATUS
 PORTS                                       NAMES
331cda446325   apache/airflow:2.8.3   "/usr/bin/dumb-init …"   3 minutes ago   Up 2 minutes (healthy)
 0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   minimal-airflow-project-airflow-webserver-1
8ea78fee90ed   apache/airflow:2.8.3   "/usr/bin/dumb-init …"   3 minutes ago   Up 2 minutes (healthy)
 8080/tcp                                    minimal-airflow-project-airflow-triggerer-1
8d423a4efdf8   apache/airflow:2.8.3   "/usr/bin/dumb-init …"   3 minutes ago   Up 2 minutes (healthy)
 8080/tcp                                    minimal-airflow-project-airflow-worker-1
ac893b9e2279   apache/airflow:2.8.3   "/usr/bin/dumb-init …"   3 minutes ago   Up 2 minutes (healthy)
 8080/tcp                                    minimal-airflow-project-airflow-scheduler-1
e1d979c8f310   postgres:13            "docker-entrypoint.s…"   18 minutes ago  Up 18 minutes (healthy)
 5432/tcp                                    minimal-airflow-project-postgres-1
595e8529c08d   redis:latest           "docker-entrypoint.s…"   18 minutes ago  Up 18 minutes (healthy)
 6379/tcp                                    minimal-airflow-project-redis-1
gitpod /workspace/minimal-airflow-project (main) $
```

Installation and configuration of Airflow

Access the environment

3 possibilities

- Run commands in the CLI.
- Use an explorer to browse the web interface.
- Use the REST API.

Installation and configuration of Airflow

Access the environment

Option 1: Launch commands in the CLI

It is possible to interact via the command line, but it must be done within a defined service of type airflow-*.

For example, to obtain information from the command airflow info, you must run:

```
docker compose run airflow-worker airflow info
```

Installation and configuration of Airflow

Access the environment

Option 2: Access the web interface

Once the cluster has started, it is possible to log in to the web interface to manage the DAGs.

The webserver is available at http://localhost:8080. Reminder: the default credentials are airflow and airflow.

We will focus on this interface in a dedicated section.

Installation and configuration of Airflow

Access the environment

Option 3: Send Requests to the REST API

It is possible to use standard tools to send requests to the REST API.

For example (to retrieve a list of pools):

```
ENDPOINT_URL=http://localhost:8080/
curl -X GET \
--user "airflow:airflow" \
"${ENDPOINT_URL}/api/v1/pools"
```

We will not be using this API today.

Question 5 - Installation and configuration of Airflow

What are the different ways to access the Airflow environment to manage workflows and tasks?

- ❑ REST API for integration with other systems
- ❑ SSH connection to an Airflow server for direct management
- ❑ Command-line interface (CLI) via the terminal
- ❑ Web user interface (UI) via a web browser

Question 6 - Celery

What is Celery?

- ❑ A relational database management system.
- ❑ A Python library.
- ❑ A distributed messaging system and an asynchronous queue.
- ❑ A web server for application deployment.

Question 7 - Celery

How does Celery communicate with Apache Airflow?

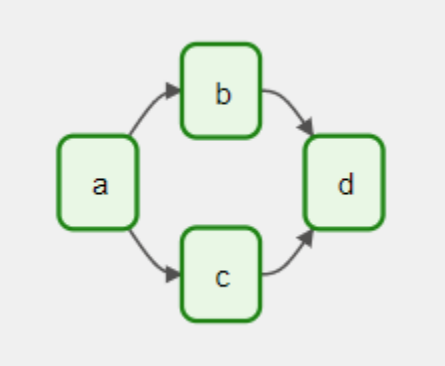
- ❑ Through REST HTTP calls.
- ❑ By writing directly to Airflow's database.
- ❑ Using messages in a queue (e.g., RabbitMQ, Redis).
- ❑ By sending operating system signals.

Creating and running a DAG

What is a DAG?

Directed Acyclic Graph

Core concept of Airflow, grouping tasks together, organised with dependencies and relationships to define how they should be executed.



This DAG defines four tasks - A, B, C and D - and dictates the order in which they must be executed, as well as the dependencies between them. It must also specify the frequency at which it should be run, for example "every 5 minutes starting tomorrow" or "daily since 1 January 2020".

The DAG itself does not care what happens inside the tasks; it is simply concerned with how to execute them - the execution order, the number of retry attempts, whether there are timeouts...

Creating and running a DAG

What is a task?

Basic unit of execution in Airflow.

Tasks are organised into DAGs (Directed Acyclic Graphs), then upstream and downstream dependencies are defined between them to express the order in which they should run.

Three basic types of tasks:

➡ Operators, predefined task templates that you can quickly assemble to build most parts of your DAGs.

➡ Sensors, a special subclass of operators that are entirely dedicated to waiting for an external event to occur.

➡ TaskFlow decorated tasks @task, which are custom Python functions packaged as tasks.

Internally, these are all subclasses of BaseOperator, and the concepts of task and operator are somewhat interchangeable, but it is useful to consider them as distinct concepts – essentially, operators and sensors are templates, and when you call one in a DAG file, you create a task.

Creating and running a DAG

Relationships between tasks:

Essential part: defining how tasks are linked to each other.

Dependencies in Airflow: upstream and downstream tasks. Which directly precedes/follows.

First, tasks are declared, then their dependencies. 2 methods:

➡ The operators >> and << (bitshift).

```
First_task >> second_task >> [third_task, fourth_task]
```

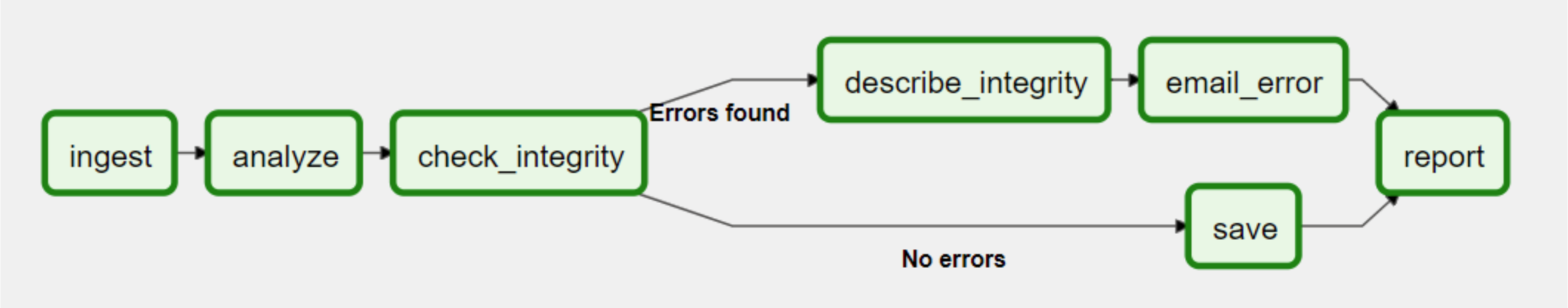
➡ The explicit methods set_upstream and set_downstream.

```
first_task.set_downstream(second_task)
third_task.set_upstream(second_task)
```

Creating and running a DAG

By default, a task runs when all its upstream (parent) tasks have succeeded, but there are many ways to modify this behaviour to add branching, to wait for only certain upstream tasks, or to change the behaviour based on the current execution history..

Creating and running a DAG



Lab

Creating and Running a DAG

Creating and running a DAG

Detail of the first DAG – importing modules

An Airflow pipeline is just a Python script that defines an Airflow DAG object. Let's start by importing the libraries we will need:

```
import textwrap
from datetime import datetime, timedelta
from airflow.models.dag import DAG
from airflow.providers.standard.operators.bash import BashOperator
# or from airflow.operators.bash import BashOperator
```

Creating and running a DAG

Detail of the first DAG – default arguments

We are going to create a DAG and some tasks, and we have the choice to explicitly pass a set of arguments to the constructor of each task (which would become redundant), or (better!) we can define a dictionary of default parameters that we can use when creating the tasks.

It is possible to override these default arguments for each task when initialising the operator.

We can define different sets of arguments that would serve different purposes (for example between a production and development environment).

```
default_args={
"depends_on_past": False,
"email": ["airflow@example.com"],
"email_on_failure": False,
"email_on_retry": False,
"retries": 1,
"retry_delay": timedelta(minutes=5),
# 'queue': 'bash_queue',
# 'pool': 'backfill',
# 'priority_weight': 10,
# 'end_date': datetime(2025, 1, 1),
# 'wait_for_downstream': False,
# 'sla': timedelta(hours=2),
# 'execution_timeout': timedelta(seconds=300),
# 'on_failure_callback': some_function, # or list of functions
# 'on_success_callback': some_other_function, # or list of functions
# 'on_retry_callback': another_function, # or list of functions
# 'sla_miss_callback': yet_another_function, # or list of functions
# 'trigger_rule': 'all_success'
},
```

Creating and running a DAG

Detail of the first DAG – DAG instantiation

We will need a DAG object to nest our tasks. Here, we pass a string that defines the dag_id, which serves as the unique identifier for the DAG. We also pass the default arguments dictionary we have just defined and set a one-day schedule for the DAG.

```
with DAG(
    "first-dag",
    default_args=default_args,
    description="Our first DAG",
    schedule=timedelta(days=1),
    start_date=datetime(2021, 1, 1),
    catchup=False,
    tags=["example"],
) as dag:
```

Creating and running a DAG

Detail of the first DAG – operators

An operator defines a unit of work to be accomplished for Airflow. Using operators is the classic approach to defining work in Airflow. For some use cases, it is preferable to use the TaskFlow API to define work in a "pythonic" context.

All operators inherit from BaseOperator, which includes all the required arguments to execute work in Airflow. From there, each operator includes unique arguments for the type of work it performs. Some of the most popular operators are PythonOperator, BashOperator and KubernetesPodOperator.

Here, we will use BashOperator to run some bash scripts.

Creating and running a DAG

Detail of the first DAG – tasks

To use an operator in a DAG, you must instantiate it as a task. Tasks determine how to execute the work of your operator in the context of a DAG.

In the following example, we instantiate the BashOperator as two distinct tasks in order to execute two different bash scripts. The first argument for each instantiation, task_id, acts as a unique identifier for the task.

```
t1 = BashOperator(
    task_id="print_date",
    bash_command="date",
)

t2 = BashOperator(
    task_id="sleep",
    depends_on_past=False,
    bash_command="sleep 5",
    retries=3,
)
```

Creating and running a DAG

Detail of the first DAG – tasks

Note that in the second task, we replace the retries parameter with 3.

The priority rules for a task are as follows:

1. ➡ Explicitly passed arguments
2. ➡ Values that exist in the default_args dictionary
3. ➡ The operator's default value, if it exists

Creating and running a DAG

Detail of the first DAG – Jinja templating

Airflow is capable of leveraging Jinja templating to provide a set of built-in parameters and macros. Airflow also provides hooks allowing the pipeline author to define their own parameters, macros, and templates.

In our DAG we barely scratch the surface of what is possible with templating in Airflow. The aim of this section is simply to inform about the existence of this feature, by showing the most common template variable: {{ ds }} (today's "timestamp").

```
templated_command = textwrap.dedent(
    """
    {% for i in range(5) %}
    echo "{{ ds }}"
    echo "{{ macros.ds_add(ds, 7)}}"
    {% endfor %}
    """
)

t3 = BashOperator(
    task_id="templated",
    depends_on_past=False,
    bash_command=templated_command,
)
```

Creating and running a DAG

Detail of the first DAG – Jinja templating

Core execution date / schedule variables:

- {{ ds }} → execution date (YYYY-MM-DD)
- {{ ds_nodash }} → execution date without dashes
- {{ ts }} → execution timestamp (ISO 8601)
- {{ ts_nodash }} → timestamp without punctuation
- {{ execution_date }} → deprecated alias (still often available)
- {{ data_interval_start }} → start of the data interval
- {{ data_interval_end }} → end of the data interval
- {{ logical_date }} → logical date of the DAG run

Creating and running a DAG

Detail of the first DAG – Jinja templating

Date offsets

- {{ yesterday_ds }}
- {{ tomorrow_ds }}
- {{ prev_ds }} / {{ next_ds }} (less commonly used / legacy-ish)
- {{ ds_add(ds, n) }} → add days
- {{ ds_sub(ds, n) }} → subtract days

Creating and running a DAG

Detail of the first DAG – Jinja templating

Run metadata

- {{ run_id }}
- {{ dag_run.conf }} → runtime config passed when triggering DAG
- {{ dag_run.external_trigger }} → whether manually triggered
- {{ is_backfill }} → if part of backfill run (if applicable in context)

Creating and running a DAG

Detail of the first DAG – Jinja templating

Task instance / execution metadata

- {{ ti.xcom_pull(...) }} → pull Xcoms
- {{ ti.try_number }}
- {{ ti.start_date }}
- {{ ti.end_date }}
- {{ ti.duration }}

Creating and running a DAG

Detail of the first DAG – Jinja templating

Parameters / user-defined values

- {{ params }} → DAG or task-level params dict
- {{ params.my_param }} → custom parameter

Creating and running a DAG

Detail of the first DAG – Jinja templating

Macros (helper functions available in templates)

- {{ macros.ds_add(ds, 7) }}{{ macros.ds_format(...) }}
- {{ macros.datetime }} → Python datetime module
- {{ macros.time }} → Python time module

Creating and running a DAG

Detail of the first DAG – Jinja templating

Timezone-aware helpers (important in Airflow 3)

- {{ ts }} is timezone-aware
- {{ data_interval_start }} and {{ data_interval_end }} are timezone-aware
- All new Airflow 3 scheduling is strongly aligned with data intervals, not raw execution_date

Creating and running a DAG

Detail of the first DAG – Jinja templating

Airflow 3 strongly encourages data_interval_start / data_interval_end instead of execution_date. So in modern DAGs, prefer:

- {{ data_interval_start }}
- {{ data_interval_end }}

instead of older patterns based on ds.

Creating and running a DAG

Detail of the first DAG – defining dependencies

```
t1 >> [t2, t3]
```

Creating and running a DAG

Detail of the first DAG – documentation

It is possible to add documentation for DAGs or each individual task.

DAG documentation currently supports only markdown, while task documentation supports plain text, markdown, reStructuredText, JSON and YAML.

DAG documentation can be written as a docstring at the beginning of the DAG file (recommended), or anywhere else in the file.

```
"""
First DAG with 3 operators.
"""

…/…

dag.doc_md = __doc__
```

Creating and running a DAG

DAG Execution – Initial Checks

To verify that your Python code contains no errors, we will run a Python command inside our Docker container. To do this, execute the following command to open the bash inside the Docker container:

```
docker exec -it <webserver_container_name> bash
```

If you need to find the name of your container, run the command:

```
docker ps
```

Once inside the bash, run the command:

```
python dags/first_dag.py
```

If all goes well, nothing will happen (in particular, no error will be raised).

Creating and running a DAG

DAG Execution – Additional Checks

Initialise the database tables:

```
docker exec -it <webserver/scheduler_container_name> airflow db migrate
```

Display the list of active DAGs (and check that the one we are interested in is there):

```
docker exec -it <webserver/scheduler_container_name> airflow dags list
```

Display the list of tasks in our DAG "first-dag":

```
docker exec -it <webserver/scheduler_container_name> airflow tasks list first-dag
```

Display the hierarchy of tasks in the DAG "first-dag":

```
docker exec -it <webserver/scheduler_container_name> airflow tasks list first-dag --tree
```

Creating and running a DAG

Execution – task by task

It is possible, especially for debugging, to run a specific task. The syntax is as follows for our steps:

```
docker exec -it <container> airflow tasks test first-dag print_date 2026-06-23

docker exec -it <container> airflow tasks test first-dag sleep 2026-06-23

docker exec -it <container> airflow tasks test first-dag templated 2026-06-23
```

And there you go, it's time to launch our first DAG!!! You can do this from the web interface, or from the following command:

```
docker exec -it <container> airflow dags trigger first-dag
```

We will go through the interface together to view the different elements related to this first DAG (checking the execution in the interface, verifying the successful execution with the execution logs, monitoring DAG executions with DAG Runs…).

Question 8 DAGs

What does a DAG represent in Apache Airflow?

- ❑ A representation of workflows composed of tasks and dependencies between them.
- ❑ A file format for storing data.
- ❑ A Python library for data analysis.
- ❑ A user interface for task visualisation.

Question 9 DAGs

What is the main characteristic of a DAG?

- ❑ It can contain cycles of dependencies between tasks.
- ❑ It can only contain Python-type tasks.
- ❑ It is limited to a single type of scheduling (for example, daily execution).
- ❑ It must be directed and acyclic, meaning it cannot contain cycles of dependencies.

02

Overview of essential features

Overview of essential features

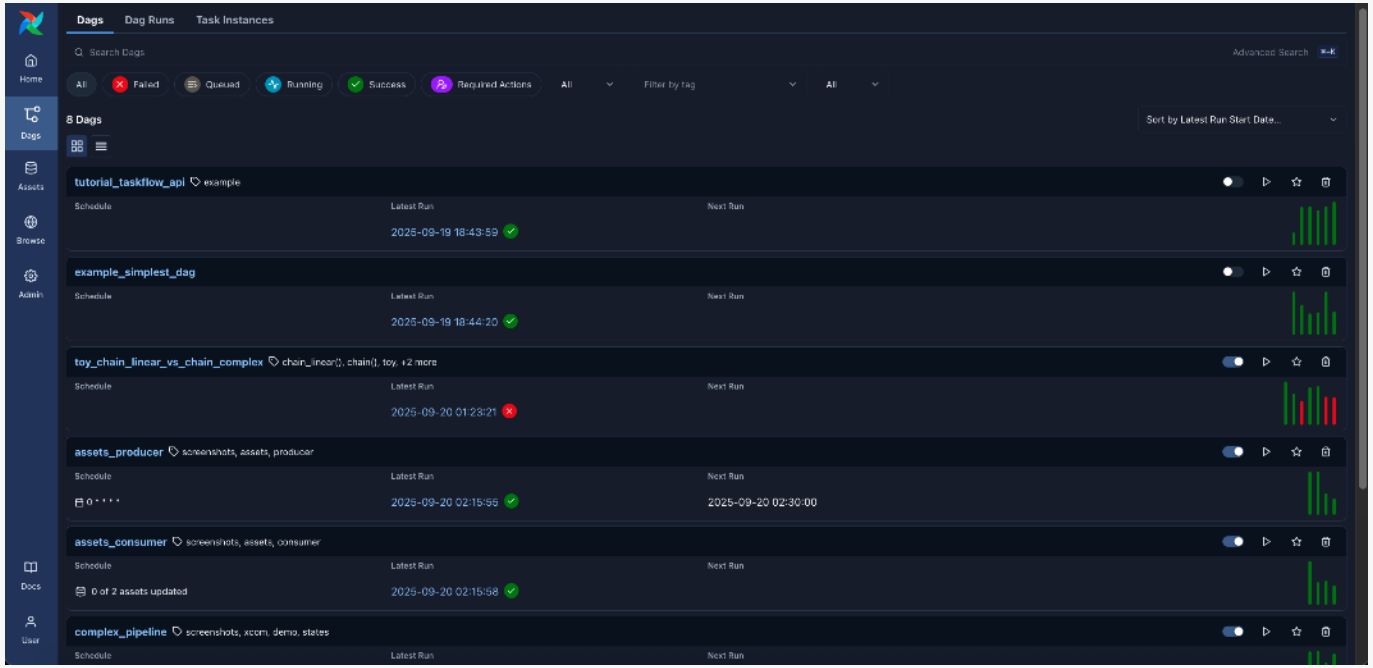
Getting started with the user interface

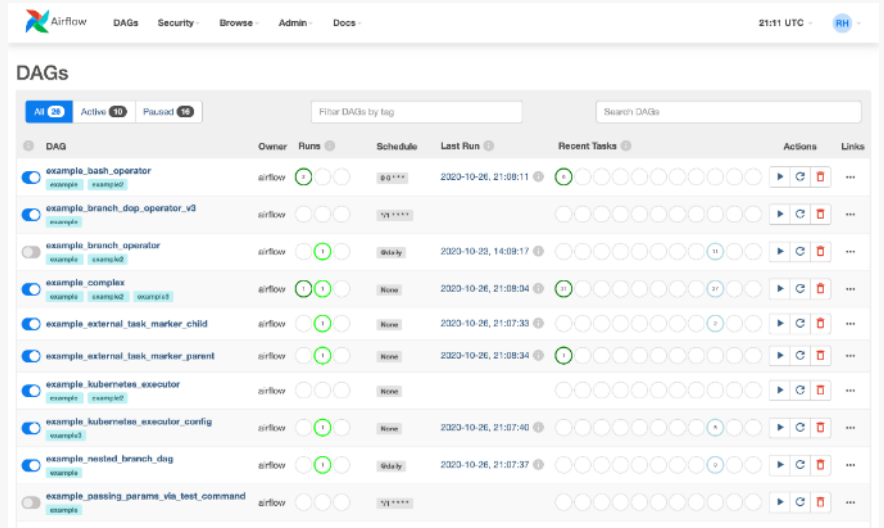
Getting started with the user interface

DAG view

Lists the DAGs in the environment, along with a set of shortcuts to useful pages. It is possible to see at a glance how many tasks have succeeded, failed, or are currently running.

To be able to filter DAGs (for example, by team), tags can be used. The filter is saved in a cookie and can be reset using the reset button.

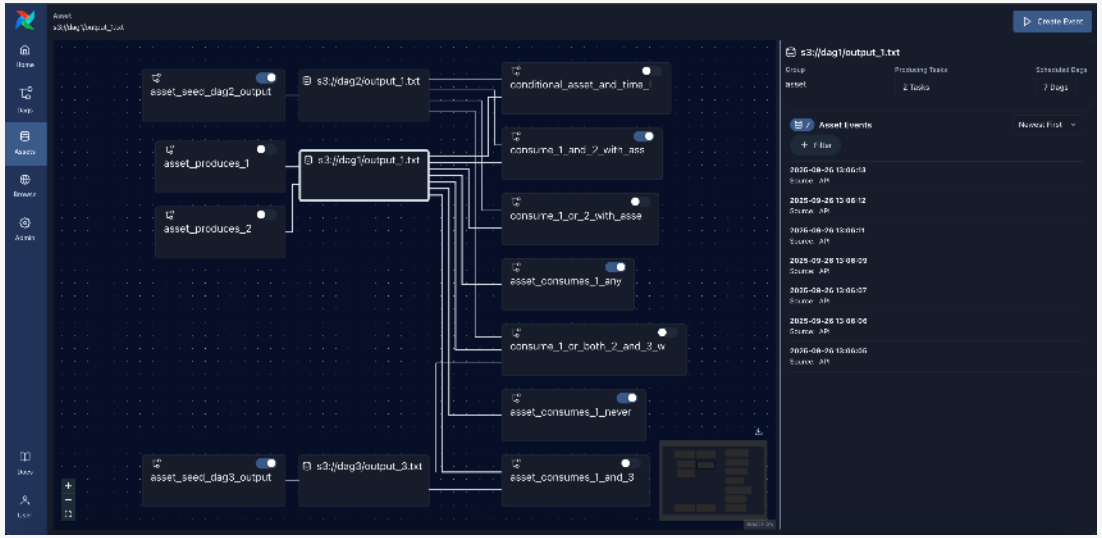


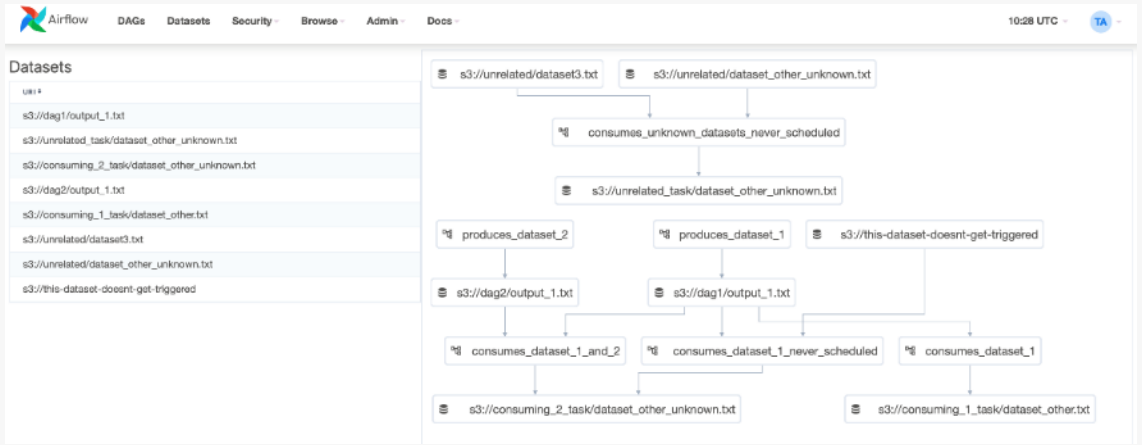



```
dag = DAG("dag", tags=["team1", "sql"])
```

Getting started with the user interface

Dataset/asset view





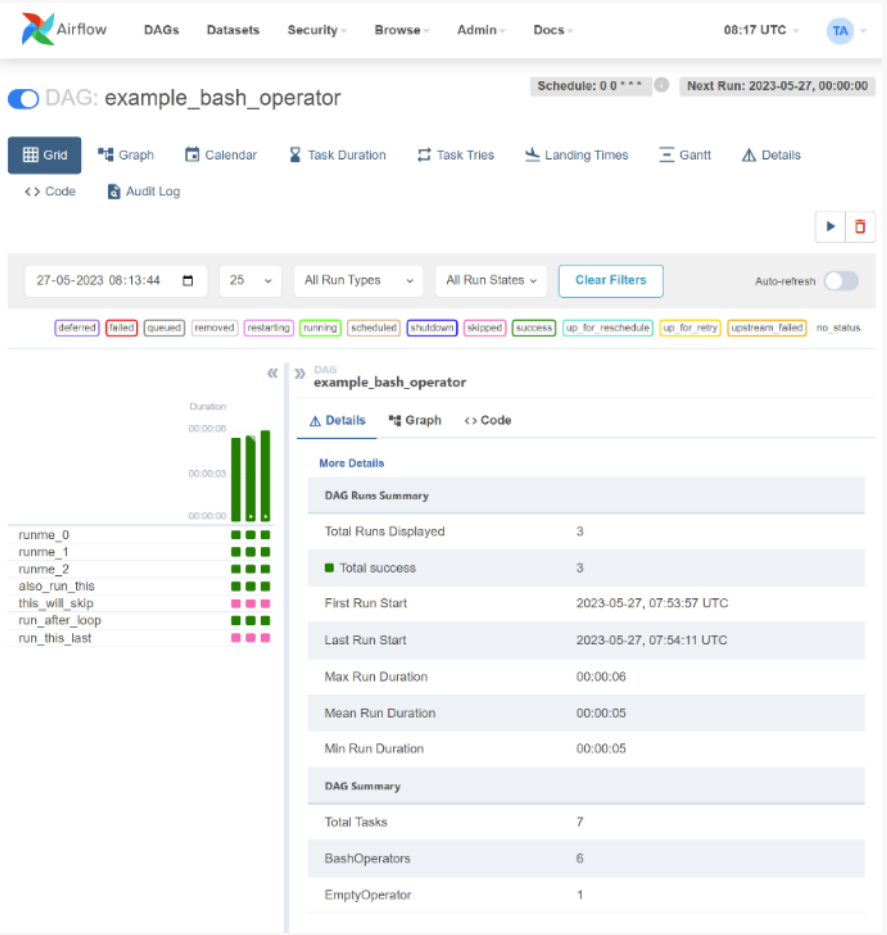
A combined list of current datasets/assets and a graph illustrating how they are produced and consumed by the DAGs.

By clicking on any dataset/asset in the list or the graph, it will be highlighted along with its relationships, and the list will be filtered to show the recent history of task instances that have updated this dataset/asset and whether this triggered other DAG runs.

Getting started with the user interface

Grid view

A bar chart and a grid representation of the DAG that extends over time. The top row is a chart of DAG runs by duration, and below are the task instances. If a pipeline is delayed, you can quickly see where the different steps are and identify those that are blocking.

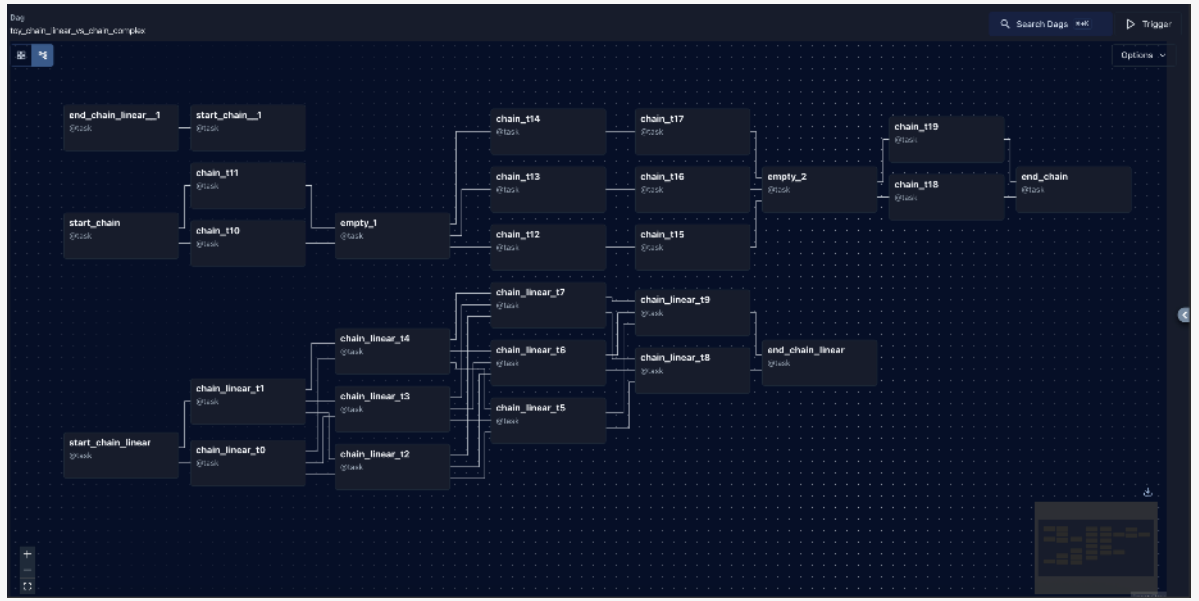


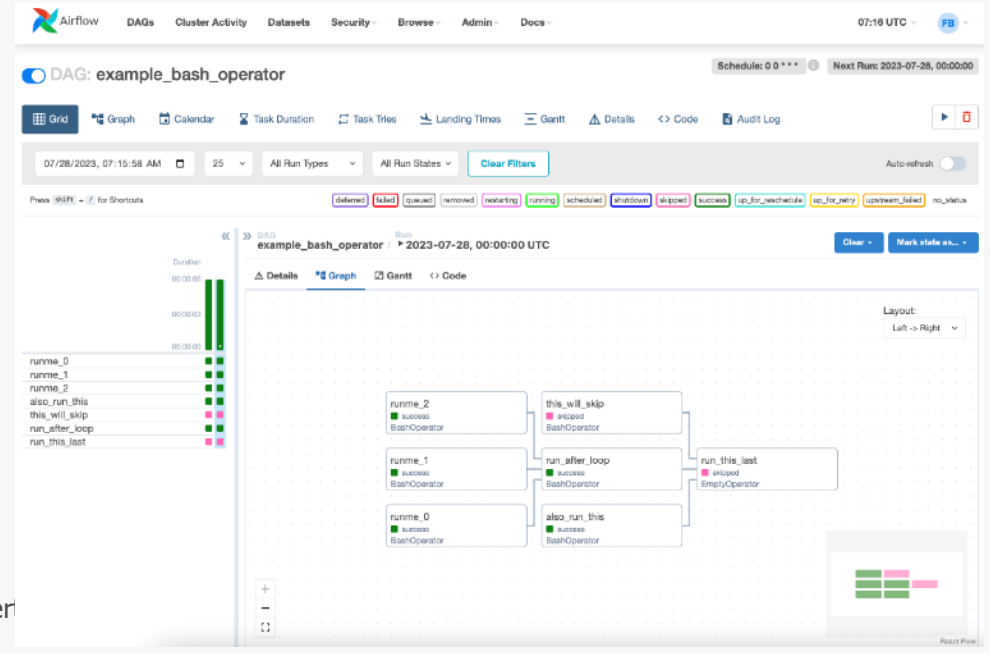
Getting started with the user interface

Graph view

Perhaps the most comprehensive view.

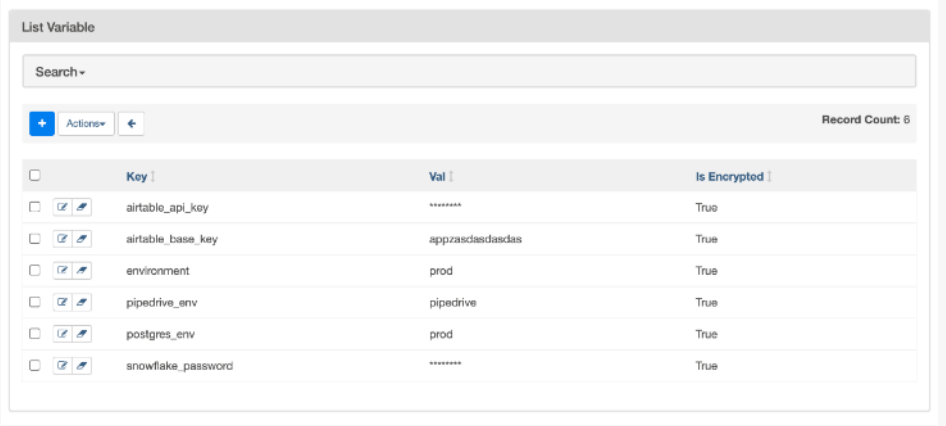
Useful for visualising the DAG dependencies and their current state for a specific run.





Getting started with the user interface

Variable view



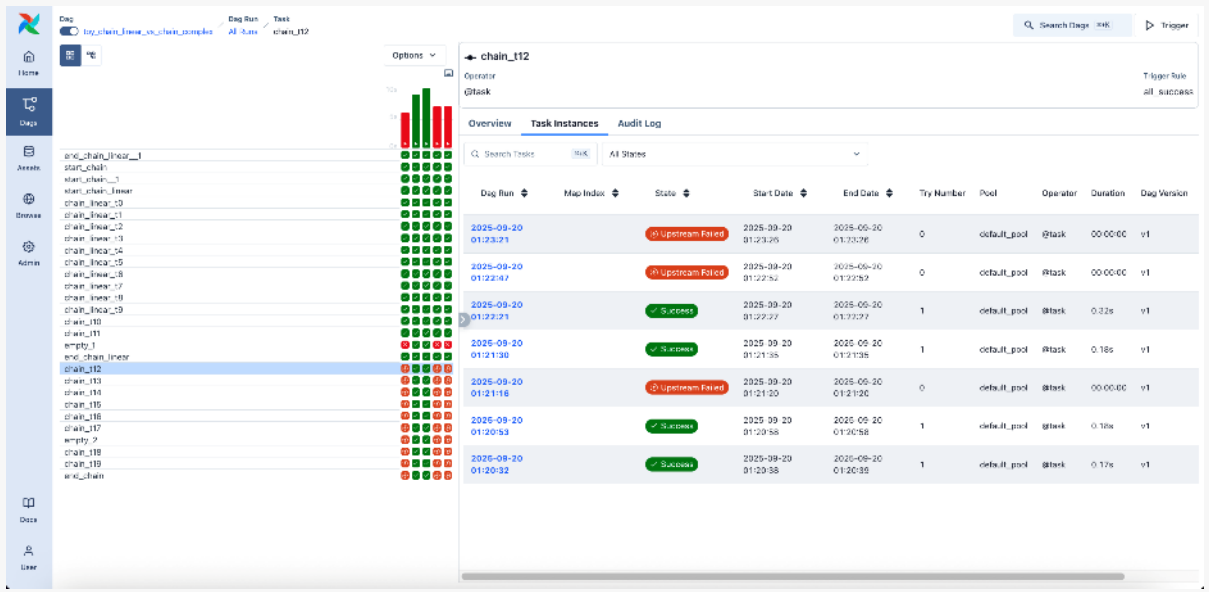
Allows you to list, create, edit or delete the key-value pair of a variable used during tasks. The value of a variable will be masked if the key contains words such as ('password', 'secret', 'passwd', 'authorization', 'api_key', 'apikey', 'access_token') by default, but can be configured to display in plain text.

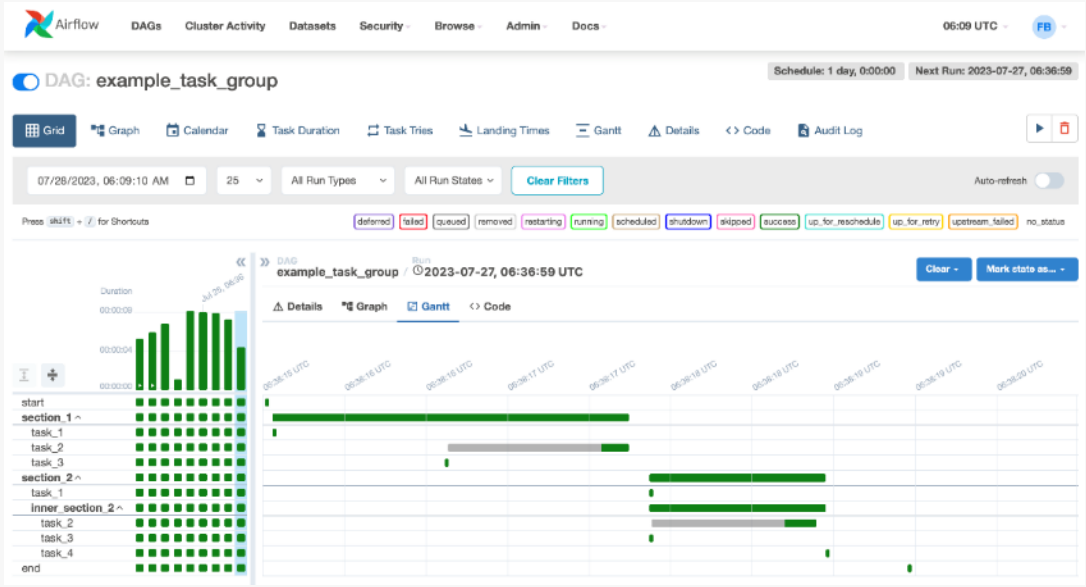
Getting started with the user interface

Gantt view

Allows analysis of task duration and overlaps.

You can quickly identify bottlenecks and where most of the time is spent for specific DAG runs.



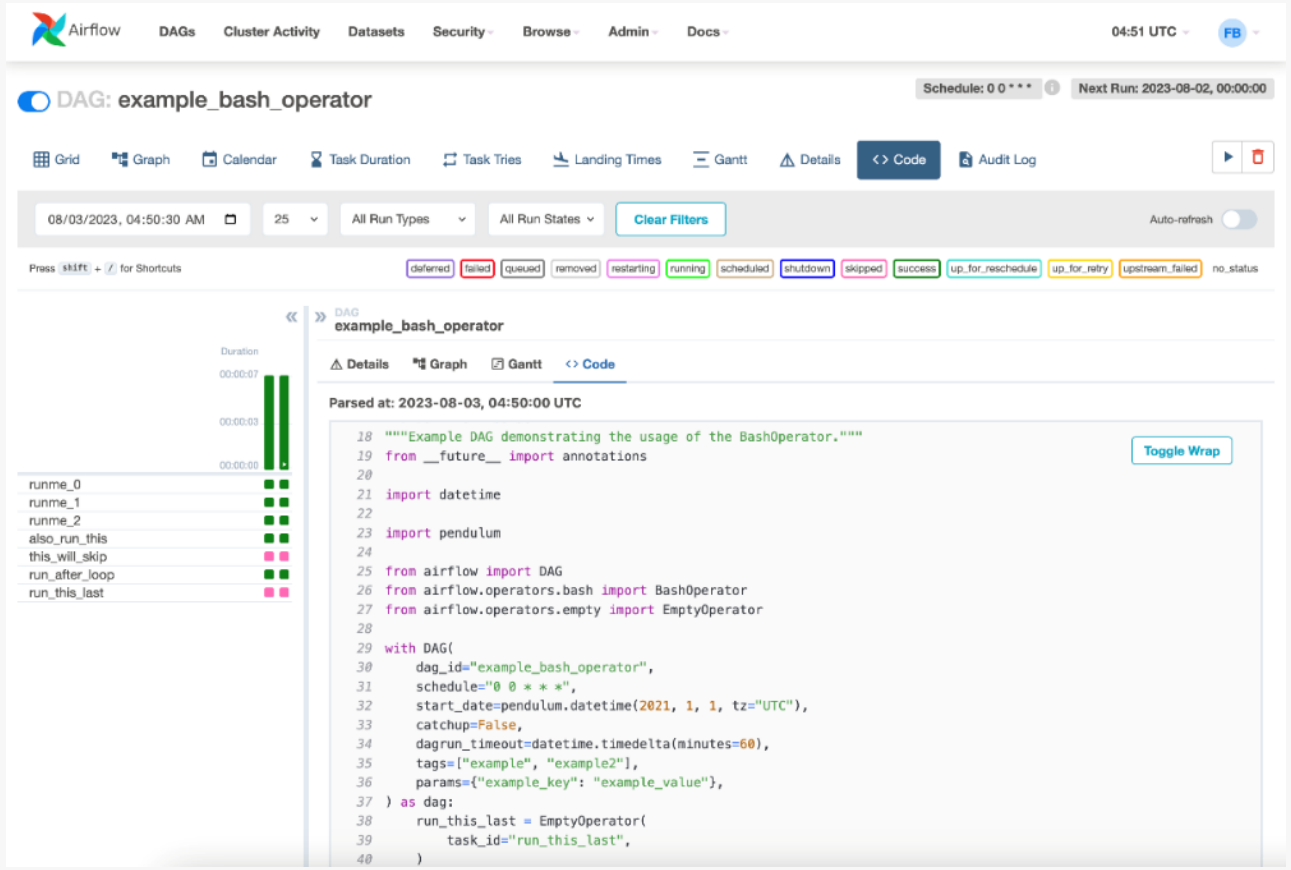


Getting started with the user interface

Code view

Quick way to access the code that generates the DAG and provide even more context (although it is typically preferable to access it from our development/version control tools).

```python
"""
First DAG with 3 operators.
"""

import textwrap
from datetime import datetime, timedelta

from airflow.models.dag import DAG
#from airflow.operators.bash import BashOperator
from airflow.providers.standard.operators.bash import BashOperator

default_args={
        "depends_on_past": False,
        "email": ["email@example.com"],
        "email_on_failure": False,
        "email_on_retry": False,
```



Question 10 - Views

What can you do in the DAG view?

- ❑ Visualise the dependencies between tasks
- ❑ See the current status of task executions
- ❑ Analyse the duration and overlap of tasks
- ❑ See the history of DAG executions

Overview of essential features - Operator management

Base Operator

Abstract base class, from which other operators are derived.

Provides the basic functionalities required to execute a task in an Airflow DAG (Directed Acyclic Graph):

- Task execution
- Dependency management
- Retry management
- Logging

Overview of essential features - Operator management

Most commonly used operators

BashOperator: Executes a bash command.

PythonOperator: Calls an arbitrary Python function.

EmailOperator: Sends an email.

SqlOperator: Executes SQL scripts directly from an Airflow DAG

TriggerDagRunOperator: Triggers the execution of another DAG

We have already implemented some of these in our first DAG.

Overview of essential features Operator management

File operators

Operators specifically designed to work with files, such as reading, writing, copying, moving…

FileSensor: Monitors a specified file or directory and waits for it to appear or disappear before continuing the execution of the DAG.

FileExistsOperator: Checks if a specified file exists in the file system before continuing the execution of the DAG.

FileToGoogleCloudStorageOperator: Copies a local file to Google Cloud Storage.

GoogleCloudStorageToBigQueryOperator: Loads data from Google Cloud Storage into BigQuery.

HiveToPigOperator: Executes a Pig script to process Hive data.

S3FileTransformOperator: Transforms data stored in Amazon S3 using a specified transformation.

Overview of key features Operator management

Sensor operators

Used to wait for a specific condition to be met before continuing the execution of a DAG.

These operators are useful when a task depends on the completion or occurrence of an external event, such as the creation of a file, the availability of a resource, or any other predefined state.

FileSensor: Waits for a specified file or directory to be created in the file system.

HttpSensor: Waits for a specified URL to be accessible via HTTP.

SqlSensor: Waits for a specified SQL query to return a certain result.

S3KeySensor: Waits for a specified key to be present in an Amazon S3 bucket.

ExternalTaskSensor: Waits for a specified task in another DAG to be completed.

TimeDeltaSensor: Waits for a certain amount of time before continuing.

Overview of essential features - Operator management

Listeners

Components that listen to events generated by the Airflow system during the execution of tasks and DAGs.

They can be used to trigger additional actions in response to these events (for example, to send email notifications when a task fails or to log information about task execution in an external log tracking system).

Often implemented as custom Python classes that inherit from the BaseDagBag base class.

Examples of events that listeners can listen to in Airflow:

on_success Triggered when a task succeeds.

on_failure Triggered when a task fails.

on_retry Triggered when a task is retried after failing.

on_execute Triggered when the task starts its execution.

on_kill Triggered when a task is manually killed.

Overview of essential TP features

Operator management

Overview of essential features - Operator management

From all the previous elements, create a DAG in Airflow that uses a BashOperator to download data from an external source and a PythonOperator to process it.

The external source will be the following:

https://raw.githubusercontent.com/CourseMaterial/DataWrangling/main/flowerdataset.csv

The processing will be the addition of a "volume" column corresponding to the product of the sepal_length and sepal_width columns.

You can use the temporary directory /tmp to store the downloaded file.

You can use a curl command.

You can set your default arguments or use the following: default_args = {

```
  'owner': 'airflow',
  'start_date': datetime(2026, 6, 23),
  'retries': 4,
  'retry_delay': timedelta(minutes=1),
}
```

Question 12 Operators

Which operator is used to execute Bash commands in Airflow?

- ❑ BashOperator
- ❑ PythonOperator
- ❑ SQLSensor
- ❑ FileSensor

Question 13 Operators

Which operator is used to execute Python commands in Airflow?

- ❑ BashOperator
- ❑ PythonOperator
- ❑ SQLSensor
- ❑ FileSensor

Overview of essential features

DAG with Sensor

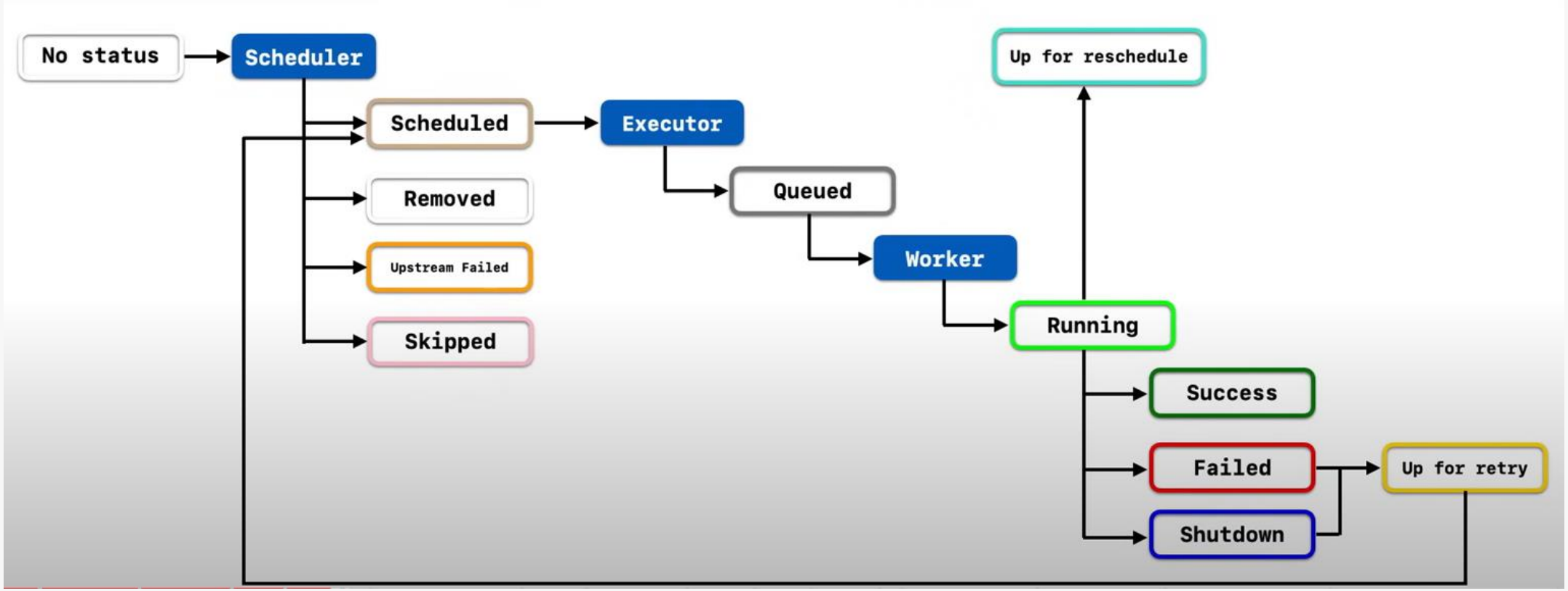
Overview of essential features

DAG for creating, inserting and deleting data in a Postgres table

03

Scheduling and workflows

Scheduling and workflows - Scheduling and planning



Scheduling and workflows - Scheduling and planning

Cron expressions

Used to specify precise scheduling times in Apache Airflow.

Follow a specific syntax based on standard cron expressions.

Allow defining recurring schedules, such as "every day at 10am" or "every hour".

* * * * * Every minute of every hour of every day of every month.

0 * * * * At the 0th minute of every hour of every day of every month.

0 10 * * * At 10am every day of every month.

0 0 * * MON At midnight every Monday.

In the context of Airflow, these expressions are used in the DAG schedule configuration to determine when tasks should be executed. For example, to run a DAG every day at midnight, you can use the expression 0 0 * * *.

Scheduling and workflows - Scheduling and planning

Start and end dates

Start Date Date from which the DAG tasks begin to be scheduled for execution. Any task scheduled before this date will not be executed, even if its start time is earlier than the start date. This allows control over when the DAG becomes active.

End Date Date until which the tasks of your DAG will be scheduled for execution. After this date, no new task execution will be scheduled, even if the task scheduling extends beyond this date. This helps to limit the execution duration of the DAG.

These two parameters are often used to control the scheduling behaviour of a DAG, by defining a time window during which tasks can be executed. This allows scheduling task execution within specific intervals and limits the lifespan of the DAG for resource management reasons or compliance with scheduling constraints.

Scheduling and workflows - Scheduling and planning

Schedule Interval

Specifies how often the tasks of a DAG should be scheduled for execution. This controls the rate at which tasks are executed by defining regular intervals between each execution.

@hourly One execution every hour.

@daily One execution every day at midnight.

@weekly One execution every week on Sunday at midnight.

@monthly One execution every month on the first day of the month at midnight.

**'/15 * * * ' : Schedules an execution every 15 minutes.

Of course, you can define custom intervals using the cron notation we have just seen.

Use scheduling intervals to adjust the frequency at which tasks are executed according to the needs of the workflow.

Scheduling and workflows - Scheduling and planning

Missed executions with Backfill

Backfill = retroactive execution or recovery of missed tasks in a DAG based on the specified start date and the configuration of the "catchup" option.

When backfilling is enabled for a DAG, Airflow checks the specified start date for that DAG and verifies if there are any missed executions between that start date and the current date. If executions are missed, Airflow runs them retroactively to fill gaps in the execution schedule.

It is possible to manage backfilling with the catchup option or manually (via command line).

Backfilling is useful when setting up new DAGs or when Airflow is offline for a period and scheduled executions have been missed. This ensures that all planned tasks are executed and data is processed correctly, even after interruptions or delays in execution.

Scheduling and workflows - Scheduling and planning

Catching up on tasks with catchup

Catchup = executing tasks for past periods when the DAG is first created or when a schedule is changed.

By default, when catchup of tasks is enabled for a DAG (catchup=True), Airflow will execute all missed tasks from the DAG's start date up to the current date. This ensures that tasks are run for all past periods when the DAG is activated, even if these are before the DAG's creation.

For example, if a DAG starts on 1 January and is run for the first time on 15 January with catchup enabled, Airflow will execute all missed tasks for the days from 1 January to 15 January.

Conversely, if catchup is disabled (catchup=False), Airflow will only execute tasks from the specified start date in the DAG, without considering past periods.

Scheduling and workflows - Scheduling and planning

Number of concurrent runs with Max Active Runs

Controls the maximum number of concurrent executions (or "runs") of a DAG. This means it limits the number of instances of the same DAG that can be running at the same time.

For example, if you set "Max Active Runs" to 1 for a specific DAG, this means Airflow will only run one instance (run) of that DAG at a time. As long as that execution is not finished or has failed, Airflow will not start another run of the same DAG, even if a trigger or schedule occurs.

This setting is useful for controlling the workload in the Airflow system and to avoid overloading resources when running many DAGs simultaneously. It should be adjusted according to the system's capabilities and the workflow requirements.

Scheduling and workflows - Scheduling and planning

Service Level Agreement (SLA, Service Level Agreement)

Allows defining time constraints for task execution within a DAG. Using SLAs in Airflow enables monitoring workflow performance and ensuring tasks are completed within the specified deadlines.

An SLA can be set for each individual task in our DAG by specifying a maximum execution time. This means the task must be finished within this allotted time from its start.

Once SLAs are defined, Airflow monitors task execution in real time. If a task fails to complete within the time specified by its SLA, Airflow marks it as having exceeded the deadline.

When a task exceeds its SLA deadline, Airflow can trigger user-defined actions, such as sending email alerts or executing recovery tasks.

Airflow provides monitoring and visualisation tools to track SLA violations. You can view metrics and reports to assess workflow performance against service level objectives.

We will not handle them in this introductory training, but it is enough to set the "sla" argument.

Scheduling and workflows - Scheduling and planning

Managing time zones with Timezone

In Airflow, time zone management is done by setting the timezone parameter at the DAG level or at the task level (this allows specifying the time zone in which the start and end times of tasks should be interpreted).

```
with DAG(
  dag_id='my_dag’,
  default_args=default_args,
  start_date=datetime(2026, 6, 23),
  schedule='@daily’,
  timezone='Europe/Paris',
) as dag:
  pass
```

Management at DAG level

```
task1 = BashOperator(
  task_id='task1’,
  bash_command='echo hello’,
  timezone='America/New_York',
)
```

Management at task level

Scheduling and workflows - Scheduling and planning

Retry Logic (1/2)

Determines the behaviour of tasks in case of failure during their execution. When a task fails, Airflow can be configured to automatically retry the execution of the task after a certain delay. This feature is useful for handling temporary errors or connectivity issues that may occur during the execution of a workflow.

```
default_args = {
  'owner': 'airflow’,
  'retries': 3, # Number of retries
  'retry_delay': timedelta(minutes=5), # Delay between each retry
  'retry_exponential_backoff': True, # Use an exponential retry delay
  'retry_exponential_backoff_delay': timedelta(seconds=10), # Initial delay for exponential retry
  'retry_on_timeout': False, # Retry on timeout
  'retry_on_upstream_retry': True, # Retry if a parent task has failed
}

dag = DAG(
  dag_id='my_dag’,
  default_args=default_args,
  schedule='@daily’,
):
  pass
```

Retry Logic at DAG level

Scheduling and workflows - Scheduling and planning

Retry Logic (2/2)

```
task1 = BashOperator(
  task_id='task1’,
  bash_command='echo Hello’,
  retries=2, # Number of retries for this task
  retry_delay=timedelta(minutes=5), # Delay between each retry
)
```

Retry Logic at task level

Scheduling and workflows - Scheduling and planning

Execution timeouts

Execution timeouts are defined at the individual task level using the execution_timeout argument when creating the task. It is given a value of type timedelta. This specifies the maximum duration for which a task can run before being automatically stopped.

Scheduling and workflows - Scheduling and planning

Triggers in scheduling with Triggers (1/2)

Triggers = features that allow the execution of a DAG to be triggered based on external events or specific conditions. Triggers can be used to start a DAG immediately or based on certain predefined conditions.

Schedule Trigger A DAG can be triggered at regular intervals defined by the schedule option when it is created.

External Trigger A DAG can be triggered by an external event, such as a REST API call or a file dropped into a specific directory.

Airflow provides the ExternalTaskSensor operator to monitor the status of another task in an external DAG and trigger the execution of the current DAG when the target task is completed.

Manual Trigger A DAG can be triggered manually from the Airflow user interface or using the command line airflow trigger_dag.

To run a DAG on demand rather than on a fixed schedule.

Scheduling and workflows - Scheduling and planning

Triggers in scheduling with Triggers (1/2)

Webhook Trigger A DAG can be triggered by an incoming HTTP request (webhook) to a specific URL.

The HttpSensor operator is used to listen for incoming HTTP requests and trigger the execution of the DAG based on the data received.

Filesystem Trigger A DAG can be triggered by changes made to a local or remote file system.

Airflow provides operators such as S3KeySensor to monitor changes in an Amazon S3 bucket or FileSensor to monitor changes in local files.

Scheduling and workflow practical work

Scheduling and planning

Scheduling and workflows - Scheduling and planning

Brief progress update:

Based on the elements seen in the first two parts, create a new DAG that simply displays a message in the terminal (echo command). Which operator will we use for this? Complete it so that it has a retry equal to 3, a retry delay of one minute. Add an owner (for the associated tag in the UI).

Scheduling and workflows - Scheduling and planning

Let's now tackle this part.

When instantiating the DAG, specify a start date (start_time) going back a few days in the past. Also specify that the scheduling interval (schedule) should be daily. For this, use the syntax we have detailed. Finally, specify that the catchup argument is False (for now).

Run the DAG and observe the result in the UI.

Delete the DAG.

Change the catchup argument to True, rerun the DAG and observe the result in the UI.

Delete the DAG again.

Scheduling and workflows - Scheduling and planning

Set the catchup parameter back to False one last time.

Launch the DAG.

Run the following command to enter the scheduler's bash:

```
docker exec -it <conteneur_scheduler> bash
```

Execute a backfill using the following command:

```
airflow dags backfill -s <start_date yyyy-mm-dd> -e <end_date yyyy-mm-dd> <dag_id>
```

Or depending on the version

```
airflow backfill create --dag-id scheduler-dag --from-date 2026-06-10 --to-date 2026-06-20
```

Observe the results in the UI.

Scheduling and workflows - Scheduling and planning

We now want to avoid running this script at the weekend.

Create a new DAG from the previous one, but use a cron syntax to add this constraint, as well as to specify the launch time at 10 am.

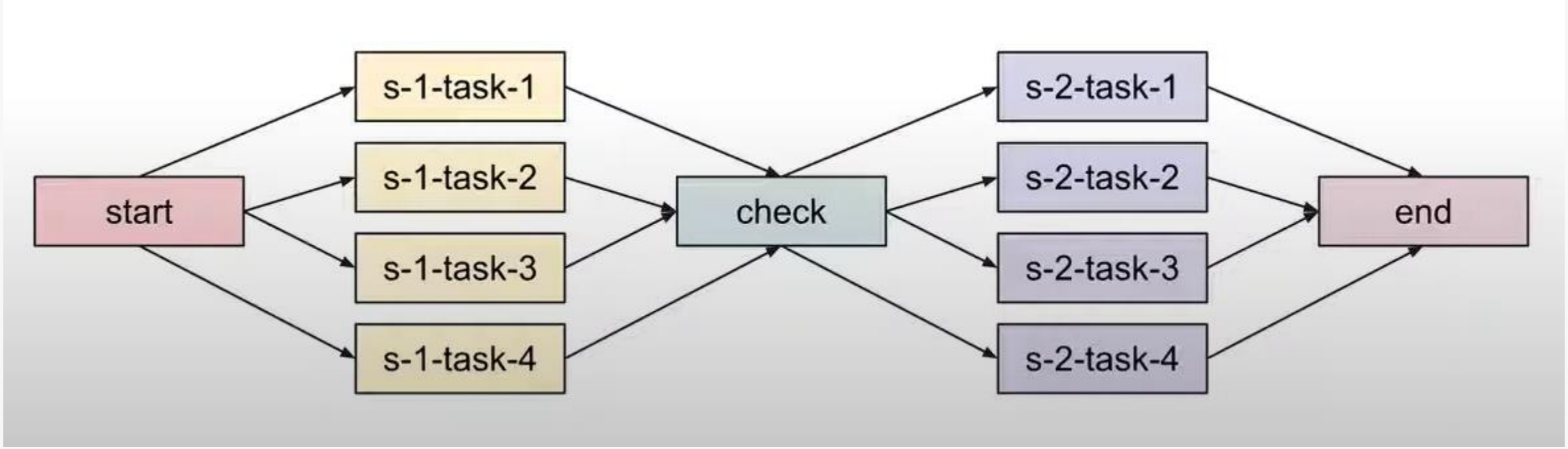
Scheduling and workflows - Scheduling and planning

Create a dag with an artificially long command (for example using the sleep command) and a timeout chosen accordingly so as to create a timeout scenario.

Launch the DAG and view the result in the UI.

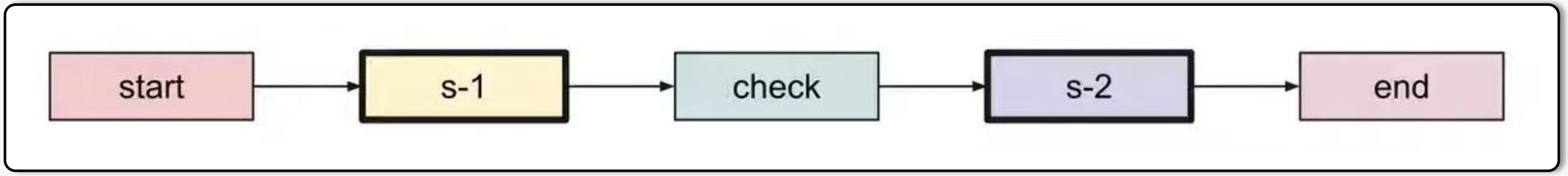
Scheduling and workflows - Workflow creation

SubDAGs / Task Grouping



Scheduling and workflows - Workflow creation

SubDAGs / Task Grouping



Scheduling and workflows - Workflow creation

Dynamic Task Generation

Useful for generating a variable number of tasks based on certain parameters or data.

The method is as follows:

➡ Define a function for the dynamic generation of tasks (this function can take parameters to customise the generation of tasks based on business logic).

➡ Inside this function, use a loop (for example, a for loop) to generate tasks based on the input data or provided parameters.

➡ For each iteration of the loop, create a new task instance using the appropriate Airflow operators (PythonOperator, BashOperator…).

➡ After generating the tasks, configure the dependencies between them.

➡ Integrate the task generation function into the DAG, using the PythonOperator to call this function and dynamically generate tasks at DAG runtime.

Scheduling and workflows - Creating workflows

Dynamic Task Generation

```
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
# or from airflow.operators.bash import BashOperator
from datetime import datetime, timedelta

# Definition of the function for dynamic task generation
def generate_tasks(task_id_prefix, num_tasks):
  tasks = []
  for i in range(num_tasks):
    task_id = f"{task_id_prefix}_{i}"
    bash_command = f"echo 'Executing task {task_id}’ »
    task = BashOperator(
      task_id=task_id,
      bash_command=bash_command,
      dag=dag,
    )
    tasks.append(task)
  return tasks
```

Scheduling and workflows - Creation of workflows

Dynamic Task Generation

```
default_args = {
  'start_date': datetime(2026, 6, 23),
  'retries': 1,
  'retry_delay': timedelta(minutes=5),
}

dag = DAG(
  'dynamic_task_generation_example’,
  default_args=default_args,
  description='Example DAG demonstrating dynamic task generation’,
  schedule=timedelta(days=1),
)
```

Scheduling and workflows - Workflow creation

Dynamic Task Generation

```
# Call the function to dynamically generate tasks
generated_tasks = generate_tasks("dynamic_task", 5)

# Final task
final_task = BashOperator(
  task_id='final_task’,
  bash_command='echo "All tasks completed« ’,
  dag=dag,
)

# Define dependencies between generated tasks
for task in generated_tasks:
  task >> final_task
```

Scheduling and workflows - Creating workflows

Branching (1/3)

Let's study the following example:

```
from airflow.providers.standard.operators.python import BranchPythonOperator
# or from airflow.operators.python import BranchPythonOperator

# Define the function for branching
def decide_branch(**kwargs):
  # Get the day of the week (0 for Monday, 6 for Sunday)
  weekday = kwargs['execution_date'].weekday()
  # If the day is a weekday (Monday to Friday)
  if weekday < 5:
    return 'weekday_task’
  # Otherwise, it is a weekend
  else:
    return 'weekend_task'
```

Scheduling and workflows - Creating workflows

Branching (2/3)

```
# Instantiate the DAG
dag = DAG(
  'dag_with_branching’,
  default_args=default_args,
  schedule='@daily',
)

# Create the branching operator
branching_task = BranchPythonOperator(
  task_id='branching’,
  python_callable=decide_branch,
  provide_context=True, # Allows the function to access context information such as the execution date
  dag=dag,
)
```

Scheduling and workflows - Creating workflows

Branching (3/3)

```
# Create tasks corresponding to the branches
weekday_task = BashOperator(
  task_id='weekday_task’,
  bash_command='echo "It\'s a weekday« ’
  dag=dag,
)

weekend_task = BashOperator(
  task_id='weekend_task’,
  bash_command='echo "It\'s a weekend« ’,
  dag=dag,
)

# Define dependencies between tasks
branching_task >> [weekday_task, weekend_task]
```

Scheduling and workflows - Creating workflows

Parameter passing

Parameter passing: use a parameter/variable type result produced by one task in another task.

For this, we use the XCom (Cross-Communication) feature, provided the data is "small" (we are not going to transmit entire tables this way, but rather write the results somewhere).

At the upstream task level, we can use the task_instance object (or ti) to access the data of the currently running task and to push data to XCom:

```
ti.xcom_push(key, value)
```

In the (function of the) downstream task, we access the passed data using this same ti:

```
ti.xcom_pull(task_ids='task_id', key='key')
```

Parameter passing is particularly useful for branching. The branching function can use parameters and decide which branch to follow based on them (and return the name of the corresponding task).

Scheduling and workflow - lab

Creating workflows

Scheduling and workflows - Scheduling and planning

Resume the DAG dag-get-ext-data. Add, after the data processing, a branching based on a random condition (to simplify), as well as tasks to display the message "Beautiful day to pick flowers" in one case, "Unfortunately, work must be done" in the other. PythonOperators or BashOperators can be used as preferred.

Scheduling and workflows - Scheduling and planning

Of course, the actual conditions do not correspond to the random drawing of a number.

We will create a branching condition that depends on the result of a previous task.

Adapt the previous DAG so that the messages after the branching are "Large dataset" if the number of rows in the dataset exceeds 1000, "Small dataset" otherwise. This can practically represent the business situation where large and small datasets would not follow the same downstream processing.

Scheduling and workflows - Scheduling and planning

Taskflow API: designed to make code simpler, cleaner, and easier to maintain. We write plain Python functions, decorate them, and Airflow handles the rest, including task creation, dependency wiring, and passing data between tasks.

Let's have a look at some examples:
https://airflow.apache.org/docs/apache-airflow/stable/tutorial/taskflow.html

04

Usage and scenarios

Airflow Fundamentals lab

Building a data pipeline

Usage and scenarios - Building a data pipeline

Let's implement the principles we have seen so far.

For this practical work, however, use the TaskFlow API to have a good representation of the range of possible syntaxes.

STEP 1

Based on the CoinGecko API documentation (https://www.coingecko.com/api/documentation), create a DAG with the following tasks:

extract_bitcoin_price Retrieves the bitcoin field from the response to a get request sent to the address providing cryptocurrency prices.
process_data Returns a dictionary formed from the usd and change fields of the previous result.
store_data Simulates data storage by logging the processed data.

Usage and scenarios Building a pipeline on data

STEP 2

Based on the CoinGecko API documentation (https://www.coingecko.com/api/documentation), adapt the DAG with the following changes:

extract_bitcoin_price Retrieves the historical market data of bitcoin on the day of the task execution date, through a GET request sent to a new endpoint. It will be necessary to access the execution date and format it.
process_data Returns a dictionary formed of the price in USD and volume in USD fields, by parsing the previous result.
store_data Unchanged.

Usage and scenarios Building a pipeline on data

STEP 3

Continue adapting the DAG:

extract_bitcoin_price Retrieves the historical market data of bitcoin on the day of the task execution date as well as the previous 9 days.
process_data Calculates the RSI associated with these data points. Returns the previous dictionary on the execution day as well as the RSI.

The RSI is given by the formula:

$$\text{RSI} = 100 - [100 \div (1 + (\text{Average gain during up periods} \div \text{Average loss during down periods}))]$$

In practice, we will not be able to execute this process without paying, due to the rate limit of CoinGecko. Replace the response with a dummy set.

Usage and scenarios Building a pipeline on data

STEP 4

Finally:

Add a task that creates a dataframe from the dates, prices and RSI.
Add a task calculating the average prices of each week from this dataframe and returning everything.

Bonus

Plugins and providers

Bonus Plugins and providers

Providers (Airflow suppliers)

Python installable packages that add features to Airflow:

- Hooks
- Operators
- Sensors
- Triggers (asynchronous)
- Cloud Integrations
- Secret backends
- System-level features

Examples: apache-airflow-providers-google, apache-airflow-providers-amazon

Installed via pip install apache-airflow-providers-xxx

Versioned and officially or community maintained

For a true full integration, create a provider.

Bonus Plugins and providers

Airflow Plugins

Allow adding custom code specific to an Airflow project, without creating a full provider.

A plugin can contain:

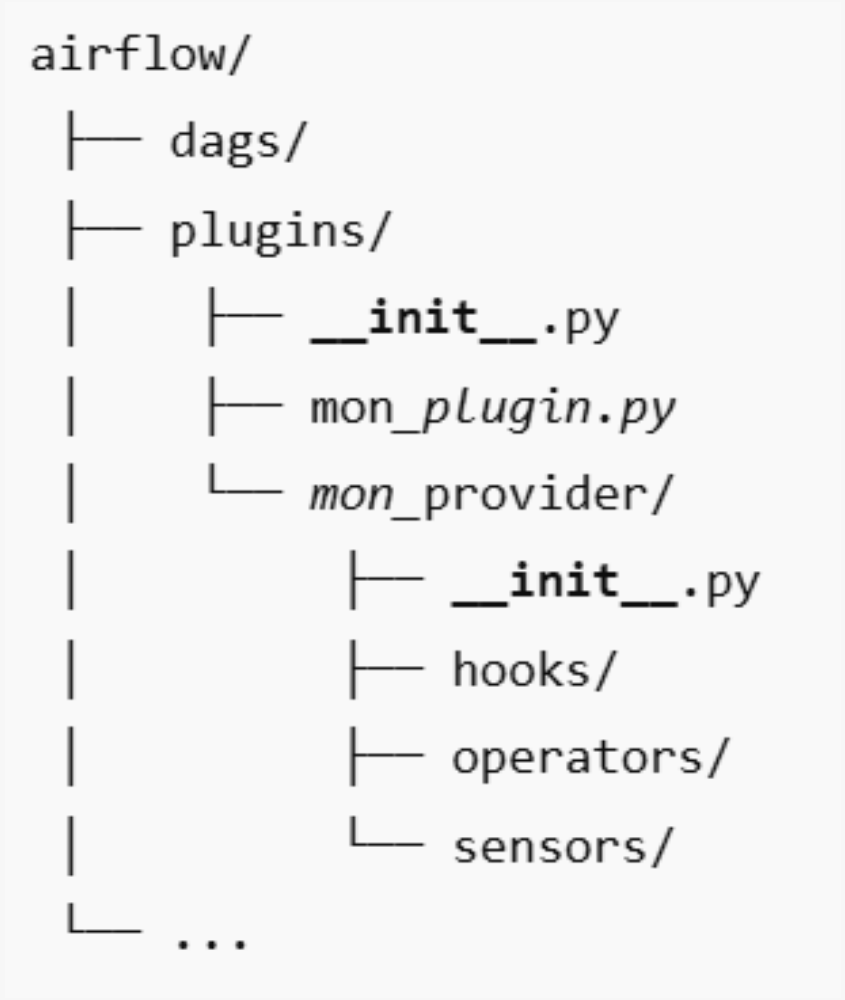
- Custom operators
- Hooks
- Sensors
- Macros
- UI additions (limited since A3)
- Flask AppBuilder Extensions

Automatically loaded at startup

Recommended for local/internal extensions

For a lightweight local extension, use a plugin.

Bonus Plugins and providers



Bonus Plugins and providers

Providers: installation

```
pip install -e .
```

```
airflow.providers.monprovider.*
```

Usage and scenarios Building a pipeline on data

STEP 5

Create a BitcoinPlugin plugin including a BitcoinAPIHook, a BitcoinExtractOperator. Adapt the DAG to use the plugin.

05

Best practices

Usage and scenarios – Airflow best practices

Developing a DAG is a three-step process:

1. Write Python code to create a DAG object,
2. Test if the code meets your expectations,
3. Configure the environment dependencies to run your DAG.

There are best practices for each of these steps.

Usage and scenarios – Airflow best practices

Minimise inter-DAG dependencies (1/2)

Divide functionalities

Split your workflows into distinct DAGs based on their functional domain. Each DAG should focus on a specific task or set of tasks and should not have direct dependencies on other DAGs.

Use external triggers

Rather than depending directly on DAGs on each other, use external triggers to initiate the execution of a DAG based on the state or execution of another DAG. This allows better isolation and less interdependence between DAGs.

Share data via external sources

Instead of sharing data directly between DAGs, use external data sources such as databases or storage systems to share information between DAGs. Each DAG can then access the necessary data without directly depending on other DAGs.

Usage and scenarios – Airflow best practices

Minimise inter-DAG dependencies (2/2)

Using Airflow variables

Use Airflow variables to share parameters or configurations between DAGs if necessary. This helps reduce direct dependencies between DAGs by avoiding the need for them to communicate directly with each other.

Avoid cyclic dependencies

Ensure there are no cyclic dependencies between DAGs, where a DAG depends on itself or on another DAG that indirectly depends on it. This can cause execution and scheduling issues.

Usage and scenarios – Airflow best practices

Reducing the complexity of your DAGs (1/5)

Although Airflow can handle many DAGs with numerous tasks and dependencies between them, when you have many complex DAGs, their complexity can impact scheduling performance. To keep your Airflow instance efficient and well-utilised, you should strive to simplify and optimise your DAGs whenever possible – remember that the process of parsing and creating DAGs is just running Python code, and it is up to you to make it as performant as possible. There are no magic recipes to make your DAG "less complex" – since it is Python code, the author of the DAG controls the complexity of their code.

There are no "metrics" for DAG complexity, in particular, there are no metrics that can tell you if your DAG is "simple enough". However, as with any Python code, you can certainly say that your DAG code is "simpler" or "faster" when it is optimised.

Nevertheless, here are some pointers:

Usage and scenarios – Airflow best practices

Reducing the complexity of your DAGs (2/5)

Make your DAG loading faster.

This is a single improvement tip that can be implemented in various ways, but it has the greatest impact on scheduler performance. Whenever you have the opportunity to make your DAG loading faster, go for it, if your goal is to improve performance.

https://airflow.apache.org/docs/apache-airflow/stable/best-practices.html

Usage and scenarios – Airflow best practices

Reducing the complexity of your DAGs (3/5)

Make the structure of your DAG simpler.

Each task dependency adds additional processing overhead for scheduling and execution. A DAG with a simple linear structure A -> B -> C will experience fewer delays in task scheduling than a DAG with a deeply nested tree structure with an exponentially growing number of dependent tasks, for example. If you can make your DAGs more linear – where at any single execution point there are fewer potential tasks to run among the tasks, this will likely improve overall scheduling performance.

Usage and scenarios – Airflow best practices

Reducing the complexity of your DAGs (4/5)

Reduce the number of DAGs per file.

Although Airflow is optimised for the case where multiple DAGs are in a single file, there are certain parts of the system that sometimes make it less performant, or introduce more delays than having these DAGs spread across multiple files. The simple fact that a file can only be parsed by one FileProcessor makes it less scalable, for example. If you have many DAGs generated from one file, consider splitting them if you find it takes a long time to reflect changes made to your DAG files in the Airflow user interface.

Usage and scenarios – Airflow best practices

Reduce the complexity of your DAGs (5/5)

Write efficient Python code.

A balance must be found between fewer DAGs per file, as mentioned above, and writing less code overall. The creation of Python files that describe the DAGs should follow programming best practices and not be treated as configurations. If your DAGs share similar code, you should not copy it over and over again into a large number of nearly identical source files, as this will lead to a number of unnecessary repeated imports of the same resources. Instead, you should aim to minimise repeated code across all your DAGs so that the application can run efficiently and can be easily debugged.

Usage and scenarios – Airflow best practices

Use templating (1/2)

Use of Jinja Templating Airflow uses Jinja Templating for rendering templates. You can use Jinja tags ({{ }}) to insert variables, expressions, filters and loops into your tasks, in operator parameters and in other parts of your Airflow code.

Airflow Variables Airflow variables can be used as templates to store and retrieve dynamic values such as file paths, configurations, or connection identifiers. You can access variables in your Airflow code using {{var.variable_name }}.

Dynamic Operator Parameters You can use templates to define dynamic parameters in Airflow operators, such as file paths, SQL queries, email addresses, etc. These parameters can be rendered dynamically based on variables or other values in your Airflow environment.

Usage and scenarios – Airflow best practices

Using Templating (2/2)

Using Macros: Airflow provides a set of built-in macros that can be used in Jinja templates to obtain information such as the current date, execution start and end dates, time offsets, etc. You can use these macros in your tasks and operator parameters for dynamic scheduling and conditional execution.

Advanced Customisation: By combining Jinja Templating with Airflow variables, macros, and conditional expressions, you can create highly customised and dynamic workflows. This allows flexible configuration of tasks, conditional dependencies, execution parameters, and other aspects of your data pipeline.

Usage and scenarios – Airflow best practices

Creating a Task Properly (1/2)

A good analogy for tasks in Airflow: transactions in a database.

This implies that you should never produce incomplete results from your tasks. An example is not producing incomplete data in HDFS or S3 at the end of a task.

It often happens that Airflow retries a task in case of failure. Therefore, tasks must produce the same result on each new execution (idempotence). In particular:

➡ Do not use INSERT during a new task execution, as an INSERT statement could result in duplicate rows in your database. Replace it with UPSERT.

➡ Read and write to a specific partition. Never read the most recent data available in a task. Someone might update the input data between new executions, leading to different outputs. A better way is to read the input data from a specific partition. You can use data_interval_start as the partition. You should also follow this partitioning method when writing data to S3/HDFS.

➡ The Python datetime now() function returns the current datetime object. This function should never be used inside a task, especially for critical calculations, as it leads to different results on each execution. (It is acceptable to use it, for example, to generate a temporary log.)

Usage and scenarios – Airflow best practices

Create a task properly (2/2)

You should define repetitive parameters such as connection_id or S3 paths in default_args rather than declaring them for each task. default_args help avoid errors such as typos. Additionally, most connection types have unique parameter names in tasks, so you can declare a connection once in default_args (e.g. gcp_conn_id) and it is automatically used by all operators that use this connection type.

Usage and scenarios – Airflow best practices

Delete a task properly

In general, be cautious when deleting a task from a DAG. You will not be able to see the task in Graph View, Grid View… which will make it difficult to check the logs of that task from the Web server. If this is not desired, please create a new DAG.

One must not forget the possibility of wanting to perform retrospective analyses.

Usage and scenarios – Airflow best practices

Communication (a bit off the programme of this introductory training, for reference)

Airflow executes the tasks of a DAG on different servers if you use the Kubernetes executor or the Celery executor. Therefore, you should not store files or configuration in the local file system as the next task is likely to run on a different server without access to it, for example a task that downloads the data file which the next task processes. In the case of the Local executor, storing a file on disk can make retries more difficult, for example if your task requires a configuration file deleted by another task in the DAG.

If possible, use XCom to communicate small messages between tasks and a good way to pass larger data between tasks is to use remote storage such as S3/HDFS. For example, if we have a task that stores processed data in S3, this task can push the S3 path for the output data into XCom, and downstream tasks can retrieve the path from XCom and use it to read the data.

Tasks should also not store authentication parameters such as passwords or tokens inside them. Wherever possible, use Connections to securely store data in the Airflow database and retrieve them using a unique connection ID.

Usage and scenarios – Airflow best practices

Python code level

You should avoid writing top-level code that is not necessary for creating operators and establishing DAG relationships between them. This is due to the design decision for the Airflow scheduler and the impact of top-level code processing speed on Airflow's performance and scalability.

The Airflow scheduler runs code outside the execute methods of operators with a minimum interval of min_file_process_interval seconds. This is done to allow dynamic DAG scheduling - where scheduling and dependencies can change over time and affect the next DAG scheduling. The Airflow scheduler continuously tries to ensure that what you have in the DAGs is properly reflected in the scheduled tasks.

More specifically, you should not perform database access, heavy computations, and network operations.

One important factor impacting DAG loading time, which can be overlooked by Python developers, is that top-level imports can take a lot of time and generate a lot of overhead, which can be easily avoided by converting them into local imports inside Python functions, for example.

Usage and scenarios – Airflow best practices

Airflow Variables (1/2)

Using Airflow variables involves network calls and database access, so based on what we just said, their use in top-level Python code for DAGs should be avoided as much as possible. If Airflow variables must be used in top-level DAG code, their impact on DAG parsing can be mitigated by enabling the experimental cache, configured with a reasonable ttl.

It is possible to freely use Airflow variables inside the execute() methods of operators, but you can also pass Airflow variables to existing operators via a Jinja template, which will delay reading the value until task execution.

Syntax:

```
{{ var.value.<variable_name> }}
```

Or, if you need to deserialize a JSON object from the variable:

```
{{ var.json.<variable_name> }}
```

Usage and scenarios – Airflow best practices

Airflow Variables (2/2)

At the top-level code, variables using Jinja templates do not generate a request until a task is running, whereas Variable.get() generates a request every time the DAG file is parsed by the scheduler if caching is not enabled. Using Variable.get() without enabling caching will result in suboptimal performance when processing the DAG file. In some cases, this may cause the DAG file to time out before it is fully parsed.

Usage and scenarios – Airflow best practices

Timetables

Still in the same vein, avoid using Airflow Variables/Connections or accessing the Airflow database at the top level of your schedule code. Access to the database should be deferred until the DAG execution time. This means you should not retrieve variables/connections as arguments during the initialization of your schedule class or have Variables/Connections at the top level of your custom schedule module.

Lab

Best practices: optimisation of "bad" DAGs

06

Integration of external databases

Integration of external databases - Configuration

It is possible to configure Airflow connections in 2 ways: via the UI or with configuration variables.

Depending on the type of database, the fields to fill in will obviously not be exactly the same. For example, the host field will be present for hosted or local DBs but not for SaaS or API services.

In our future examples, a Postgresql database is handled.

In the UI, it is possible to give Host the name of the postgres service from the docker-compose file or localhost with Linux, host.docker.internal with Docker Desktop/Windows/Mac OS.

The other fields are self-explanatory.

Especially when using Docker, the port must have been opened.

Integration of external databases - Writing

PostgresOperator

```
create_table = PostgresOperator(
  task_id='create_table',
  postgres_conn_id='postgre_dwh',
  sql = """
    CREATE TABLE IF NOT EXISTS airflow_test (
    id SERIAL PRIMARY KEY,
    name VARCHAR NOT NULL,
    type VARCHAR NOT NULL,
    date DATE NOT NULL,
    OWNER VARCHAR NOT NULL);
  """
)
```

Integration of external databases - Reading and manipulation

To read data from a PostgreSQL table and manipulate it (for example with pandas), a Hook is used.

It is perfectly possible to combine a PostgresOperator for pure SQL operations, and a PythonOperator containing a Hook for data processing.

Integration of external databases - Reading and manipulation

The PostgresHook is an interface to connect to a PostgreSQL database. It facilitates the execution of SQL queries and the retrieval of results.

Import:

```
from airflow.providers.postgres.hooks.postgres import PostgresHook
```

Instantiation:

```
pg_hook = PostgresHook(postgres_conn_id='postgres_default’)
```

Integration of external databases Reading and manipulation

The PostgresHook is an interface to connect to a PostgreSQL database. It facilitates the execution of SQL queries and the retrieval of results.

Main methods and attributes (1/2):

get_conn() Establishes the connection to the PostgreSQL database and returns the connection object.

```
conn = pg_hook.get_conn()
```

get_records() Executes an SQL query and returns all results as a list of tuples.

```
sql_query = "SELECT * FROM my_table"
records = pg_hook.get_records(sql=sql_query)
```

get_first() Executes an SQL query and returns the first result.

```
first_record = pg_hook.get_first(sql=sql_query)
```

Integration of external databases Reading and manipulation

Main methods and attributes (2/2):

run() Executes an SQL query without returning results.

```
insert_query = "INSERT INTO my_table (col1, col2) VALUES (%s, %s)"
pg_hook.run(sql=insert_query, parameters=('value1', 'value2'))
```

insert_rows() Inserts multiple rows into a table in a single command.

```
rows = [(1, 'value1'), (2, 'value2')]
pg_hook.insert_rows(table="my_table", rows=rows)
```

Integration of external databases Reading and manipulation

Cursor

The PostgresHook uses cursors to interact with the PostgreSQL database.

A cursor is an object used to execute SQL queries and retrieve results.

Creating and using a cursor

```
pg_hook = PostgresHook(postgres_conn_id='postgres_default')
conn = pg_hook.get_conn()
cursor = conn.cursor()
```

Executing SQL queries with the cursor and retrieving results

```
cursor.execute("SELECT * FROM my_table")
records = cursor.fetchall()
print(records)
```

Integration of external databases Reading and manipulation

Cursor

Data insertion

```
insert_query = "INSERT INTO my_table (col1, col2) VALUES (%s, %s)"
cursor.execute(insert_query, ('value1', 'value2’))
conn.commit()
```

Closing the cursor and the connection

```
cursor.close()
conn.close()
```

Integration of external databases - Case study examples

We will go through some DAGs related to integration with external databases.

Lab

DAG setup
Create a DAG with a name of your choice. Configure it to run daily at 12:00. Set a fixed start and end date Enable catchup so all historical runs are executed. Configure retries and retry delay in default settings. Set a reasonable SLA and task timeout.

First task: create metadata table in Postgres
Create a task that ensures a Postgres table for DAG runs exists and create it if not. The table must store DAG execution metadata. It must include:

- execution date
- DAG identifier

Ensure the combination of these fields is unique (primary key). Use the Postgres connection named postgres_localhost

Second task: clean existing run data
Create a task that removes any existing record for the current DAG run. The deletion must be scoped to:

- current execution date
- current DAG identifier

This ensures idempotent behavior for repeated runs.

Third task: insert current run metadata
Create a task that inserts a new record into the metadata table. The record must contain:

- execution date of the run
- DAG identifier

Values must be dynamically resolved at runtime using Airflow context

Fourth task: extract and process data
Create a Python-based task that:

- connects to the same Postgres database
- retrieves all records from the metadata table
- loads the result into a tabular data structure for analysis
- prints the full dataset
- filters the dataset for a specific DAG identifier
- prints the filtered subset

Don’t forget to define the task dependencies

07

Scheduling and advanced planning

Scheduling and advanced planning SLA

SLA (service level agreement): expectation regarding the maximum time within which a task must be completed relative to the start time of the DAG execution.

If a task takes longer than this to execute, it will then be visible in the "SLA Misses" section of the user interface, as well as sent by email with all tasks that missed their SLA provided the SMTP server has been configured and email parameters set in the DAG (we will not do this).

Tasks exceeding their SLA are not cancelled - they are allowed to run until completion.

To cancel a task after a certain execution time, timeouts must be used.

➡ To set an SLA for a task, a datetime.timedelta object is passed to the sla parameter of the task or operator. An sla_miss_callback can also be provided which will be called when the SLA is missed, to execute specific logic.

It is possible to completely disable SLA checking afterwards by setting check_slas = False in the [core] Airflow configuration.

Scheduling and advanced planning - Deadline Alerts

Airflow 3 migrates from SLA to Deadline Alerts:

https://airflow.apache.org/docs/apache-airflow/stable/howto/sla-to-deadlines.html

Scheduling and advanced planning - To go further: configure email sending

https://airflow.apache.org/docs/apache-airflow/stable/howto/email-config.html

Scheduling and advanced planning - Zombie tasks

No system works perfectly, and task instances will inevitably fail from time to time.

Airflow detects two types of task/process mismatches:

Zombie tasks are task instances stuck in a running state despite the inactivity of their associated jobs (their process has not sent a recent heartbeat because it was killed, or the machine crashed).

Airflow periodically finds them, cleans them up, and fails or retries the task depending on its settings.

Zombie tasks are tasks that are not supposed to be running but are (they are often caused by manual modification of task instances via the user interface).

Airflow periodically finds them and terminates them.

Scheduling and advanced planning - Environment variables related to a zombie task

```
export AIRFLOW__SCHEDULER__LOCAL_TASK_JOB_HEARTBEAT_SEC=600

export AIRFLOW__SCHEDULER__SCHEDULER_ZOMBIE_TASK_THRESHOLD=2

export AIRFLOW__SCHEDULER__ZOMBIE_DETECTION_INTERVAL=5
```

08

Creation and management of complex workflows

Creation and management of complex workflows - Versioning

Version control of DAGs in Apache Airflow is crucial to ensure that changes made to DAGs are tracked, tested and deployed in a controlled manner.

What can be done in practice?

- Use a version control system (Git) to store the DAGs in a repository and track changes through commits.
- Version the DAGs in the code

```
from airflow import DAG
from datetime import datetime, timedelta

VERSION = '1.0.0'

with DAG(
  dag_id=f'dag_with_version_{VERSION}',
) as dag:
  pass
```

Creation and management of complex CI/CD workflows

Integrating Airflow into a CI/CD deployment pipeline (e.g., GitHub Actions, Jenkins, GitLab CI) obviously allows the automation of tests and DAG deployments.

```
name: Deploy DAGs

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v2

      - name: Set up Python
        uses: actions/setup-python@v2
        with:
          python-version: "3.8"

      - name: Install dependencies
        run: pip install apache-airflow

      - name: Deploy DAGs
        run: |
          scp -r dags/ user@your-airflow-server:/path/to/airflow/dags/
```

Creation and management of complex workflows - Study of a DAG

Link between versioning and branching

Creation and management of complex workflows - Priority weights

The priority weights (priority_weight) of tasks in Airflow are used to manage the order of task execution when resources are limited.

When a DAG contains several tasks that can be executed in parallel, Airflow uses priority weights to decide which tasks to execute first.

This is particularly useful in environments where computing resources (such as worker slots, cluster capacity, etc.) are limited.

Of course, tasks with the highest priority weights will be executed before those with lower weights, provided they are competing for the same resources.

Creation and management of complex workflows - Deferrals

Deferred or deferrable operators in Airflow are designed to improve resource efficiency by not consuming worker slots while waiting for a condition to be met. Deferred tasks use triggers to wake up when the condition is fulfilled, allowing Airflow to manage pauses and waits more efficiently.

A deferrable operator is written with the ability to suspend itself and release the worker when it knows it must wait, and to entrust the resumption to something called a trigger. Therefore, when it is suspended (deferred), it does not take up a worker slot and the cluster spends far fewer resources on inactive operators or sensors.

In practice, triggers are small pieces of asynchronous Python code designed to be run all together in a single Python process; since they are asynchronous, they can all coexist efficiently.

Using deferrable operators as a DAG author is almost seamless; writing them, however, requires a bit more work.

#Resource_efficiency #Scalability

Creating and managing complex workflows - Going further with deferrals

To learn how to write your own deferrals (somewhat unusual):

https://airflow.apache.org/docs/apache-airflow/3.1.3/authoring-and-scheduling/deferring.html

Creating and managing complex workflows - Priority weights and deferrals

Create a DAG starting with a task featuring a 2-minute TimeDeltaSensor, then two parallel tasks, one high priority, one low priority, simulated by PythonOperators calling a dummy_task_function simply logging the task performed at INFO level.

Notice the behaviour of the TimeDeltaSensor in the past.

09

Best practices again

Best practices Minimise inter-DAG dependencies

Why minimise inter-DAG dependencies?

Reduced complexity Dependencies between DAGs can make the workflow difficult to understand and maintain. Every change in a DAG can impact other DAGs, thus increasing management overhead.

Failure isolation If a DAG fails, inter-DAG dependencies can propagate this failure to other DAGs. Keeping DAGs independent limits the impact of failures.

Ease of debugging Independent DAGs are easier to debug, as it is simpler to understand the data flow and diagnose issues without having to consider complex interactions with other DAGs.

Scalability Data pipelines can be easily scaled when DAGs are independent. Inter-DAG dependencies can create bottlenecks and contention points.

Best practices Minimise inter-DAG dependencies

Strategies to minimise inter-DAG dependencies (1/2)

Using sensors Sensors allow the creation of lightweight synchronization points without establishing direct dependencies. For example, an ExternalTaskSensor can be used to wait for the completion of a task in another DAG without creating a direct dependency.

Centralising shared data Centralised data storage systems such as databases, data lakes, or message queues can be used to share data between DAGs. Each DAG can read from or write to these systems independently. Messaging systems can be used to orchestrate events between DAGs without establishing direct dependencies.

Passing metadata To share information between DAGs, consider using a central database or a metadata service to store and retrieve this information.

Best practices Minimise inter-DAG dependencies

Strategies to minimise inter-DAG dependencies (2/2)

Breaking down complex tasks If a workflow is too complex, consider breaking it down into several smaller, independent sub-DAGs or DAGs that can be run autonomously.

Use local tasks Rather than triggering tasks in other DAGs, try to manage as much logic as possible within a single DAG using dependencies between tasks.

Share common code If multiple DAGs require similar logic, extract this logic into reusable Python functions or modules and share them between the DAGs.

Use DAG triggers DAG triggers can be useful in some cases, but use them judiciously and avoid excessive use, as this can complicate dependency management.

Use Airflow variables Airflow variables can be used to share data between DAGs securely and efficiently.

Best practices Use templates

Templates = make DAGs more dynamic, reusable and easier to maintain.

Dynamic task parameterisation Dates, file paths, table names...

More generally

Code reuse Extract it into functions, classes, use templates for variable parameters.

Centralisation of configuration Centralise repetitive configuration, such as database connections, email addresses, file paths... using templates. This facilitates maintenance by avoiding code duplication.

Use of built-in template operators

Template validation

And of course, document all this well!

https://airflow.apache.org/docs/apache-airflow/stable/templates-ref.html

Best practices - Avoid circular dependencies

A circular dependency occurs when two or more tasks depend on each other, creating an infinite loop that prevents the correct execution of tasks.

It’s common sense: avoiding circular dependencies in Airflow is crucial to ensure the stability and reliability of workflows.

And for that:

- Plan the workflow well
- Break down complex tasks
- Check that the DAG is indeed a DAG, particularly by using Airflow visualisations
- Create sub-DAGs
- Use sensor and trigger operators (rather than creating direct dependencies between tasks in different DAGs)
- Avoid inter-DAG dependencies
- Analyse the logs and more generally do monitoring

Best practices - Parallelise tasks

Properly parallelise what can be!

Do not code tasks as sequential if they are not: this deprives you of an optimisation.

Best practices - Parallelise tasks and manage concurrency

Configure parameters related to parallelisation, in particular the maximum number of tasks and DAGs executed concurrently (max_active_tasks_per_dag and max_active_runs).

```
[core]
max_active_tasks_per_dag = 16
```

```
from airflow import DAG
from datetime import datetime

default_args = {
  'owner': 'airflow’,
  'depends_on_past': False,
  'email_on_failure': False,
  'email_on_retry': False,
  'retries': 1,
}

dag = DAG(
  'example_dag’,
  default_args=default_args,
  description='A simple tutorial DAG’,
  schedule_interval='@daily’,
  start_date=datetime(2026, 6, 23),
  catchup=False,
  max_active_runs=3 # Limit to 3 active DAGs
)
```

Best practices - Parallelise tasks and manage concurrency

Use task pools

Pools allow limiting the number of tasks running concurrently for certain types of resources. They can be defined in the Airflow user interface or in the airflow.cfg file

```
airflow pool -s my_pool 10 "Description of the pool"
```

Then, assign pools to tasks to manage resources.

Best practices - Parallelise tasks and manage concurrency

Use task pools

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from datetime import datetime

with DAG() as dag:

  task1 = BashOperator(
    task_id='task1’,
    bash_command='echo "Task 1"',
  )

  task2 = BashOperator(
    task_id='task2’,
    bash_command='echo "Task 2"',
    pool='my_pool’
  )

  task3 = BashOperator(
    task_id='task3’,
    bash_command='echo "Task 3"',
    pool='my_pool'
  )

  task4 = BashOperator(
    task_id='task4’,
    bash_command='echo "Task 4"'
  )

task1 >> [task2, task3] >> task4
```

Best practices - Parallelise tasks and manage concurrency

Optimise the Airflow cluster

The Airflow scheduler can (and should) be configured to efficiently manage a large number of tasks. To do this, the parameters scheduler_heartbeat_sec, min_file_process_interval, and dag_dir_list_interval in airflow.cfg are mainly adjusted.

```
[scheduler]
scheduler_heartbeat_sec = 5
min_file_process_interval = 30
dag_dir_list_interval = 60
```

Best practices - Manage periodicity to avoid overloads

It might be overlooked but, ultimately, managing periodicity is essential to avoid system overloads and ensure workflows run smoothly and efficiently.

- Use appropriate scheduling intervals (avoid overly frequent scheduling)
- Use catchup wisely
- Use time windows

Best practices - Avoid long waiting times and blockages

First, optimise individual tasks (code design, use of caches, elimination of redundant operations, optimisation of SQL queries…)!

Correctly configure the performance parameters discussed (parallelism, dag_concurrency, max_active_runs, concurrency…).

Once this is done, optimisation will come from broader classic specific elements beyond the Airflow world. For example, use indexes on databases to help speed up read and write operations, especially on large databases.

Best practices - Last but not least, plan for failure recovery

Every system is prone to failure.

First, from a business perspective, carefully consider the scenario to follow in case of failure.

On the Airflow side, implement a retry logic, manage catchup, use time sensors but also status sensors.

In case of high criticality (to be used sparingly), possibly consider scheduling redundant executions. In this case, pay particular attention to:

- the potential non-idempotence of the processes,
- resource overload,
- additional costs,
- increased complexity of the Airflow project.

Coming back to your questions

Timetables: CronDataIntervalTimetable value storage

Where does Airflow's CronDataIntervalTimetable store its computed schedule values. In the PostgreSQL metadata tables or elsewhere?

CronDataIntervalTimetable does not store any computed schedule values in PostgreSQL or any other persistent storage.

It is a pure Python object used by the Airflow scheduler at runtime. When the scheduler needs to know when the next DAG run should occur, it constructs (or reconstructs) the timetable from the DAG definition and computes the next execution times on demand using the cron expression.

Because of this design, the timetable itself is not stateful and not persisted. It exists in memory only while the scheduler is evaluating schedules.

What is stored in the PostgreSQL metadata database are the results of the scheduling process, specifically entries in the dag_run table. These records represent actual DAG executions, including their run IDs, logical dates, and data intervals.

In practice, the flow looks like this:

1. Airflow loads the DAG definition.
2. It builds a timetable object in memory.
3. The scheduler queries the timetable for the next run time and data interval.
4. Once a run is created, only that dag_run information is persisted in PostgreSQL.

Listening to a file/folder with Airflow

1. Add the path to listen to as a volume in your docker-compose.yml, for example:

```
- ${AIRFLOW_PROJ_DIR:-.}/watch:/opt/airflow/watch
```

2. Down and up.

3. Configure a File (path) connexion:

- Connexion ID: fs_default
- Connexion type: File (path)
- Extra fields > Path: /opt/airflow

4. Write a DAG with a FileSensor to listen to a « trigger.txt » file in your watch folder.

5. Trigger it, observe, add a file, observe.

6. Try to listen to the whole folder.

7. Try to use deferrable=True.

How to install a plugin

0. Check which plugins and libraries are installed. Verify that the astronomer-cosmos plugin is not installed.

1. Create the following Dockerfile in your root directory:

```
FROM apache/airflow:3.1.3
USER airflow
RUN pip install astronomer-cosmos
```

2. In your docker-compose.yaml file:

- Comment out this line: `image: ${AIRFLOW_IMAGE_NAME:-apache/airflow:3.1.3}`
- Uncomment this line: `build: .`

3. Down and up.

4. Verify that the astronomer-cosmos plugin is installed.

Note: You could also add `_PIP_ADDITIONAL_REQUIREMENTS=astronomer-cosmos` to your .env file, but this is not considered best practice compared to the approach described above.

End of this section

Thank you!